



Documentação técnica

LPS — Plataforma de Gestão do Trabalho

Para desenvolvedores · Gerado em 29/09/2026 a partir de `README.md` e `docs/*.md`
Repositório: github.com/Lucasrafael703/workflow

Sumário

—	Visão geral	3
—	Telas da aplicação	5
01	Arquitetura	17
02	Ambiente local	20
03	Configuração	23
04	Modelos de dados	26
05	Autorização	31
06	Atividades e tarefas	34
07	Contas e autenticação	39
08	Notificações e auditoria	41
09	Rotas	44
10	Frontend	48
11	Testes	50
12	Deploy no Render	52
13	Pendências conhecidas	54

Visão geral

Aplicação Django que implementa o núcleo D0 da LPS: uma plataforma de gestão do trabalho **multi-organização** que organiza atividades e tarefas, torna filas e responsabilidades visíveis, registra o que acontece durante a execução e transforma esse histórico em informação para gestão.

- O **produto** (regras de negócio e telas) está em [Regras/](#) e [Telas/](#).
- O **código** (como essas regras foram implementadas) está documentado em [docs/](#).

Stack

Item	Versão / escolha
Framework	Django 5.2 (LTS)
Python	3.14 no desenvolvimento local · 3.12.7 no Render
Banco	SQLite (dev) · PostgreSQL via <code>DATABASE_URL</code> (prod)
Servidor	<code>runserver</code> (dev) · gunicorn + whitenoise (prod)
Frontend	Templates Django + CSS/JS estáticos, sem bundler
Dependências	<code>django-environ</code> , <code>nh3</code> (sanitização de HTML), <code>gunicorn</code> , <code>whitenoise</code> , <code>psycopy</code> — ver requirements.txt

Início rápido (Windows / PowerShell)

```
python -m venv .env
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env          # e defina SECRET_KEY
python manage.py migrate
python manage.py seed_lps_demo  # organização "Biasi", setores, ações e perfis
python manage.py createsuperuser
python manage.py runserver
```

Antes de usar as telas, **vincule seu usuário a uma organização** (Admin do Django → Accounts → Perfis → campo "organização"); sem isso toda tela redireciona para o perfil. Detalhes em [docs/02_AMBIENTE_LOCAL.md](#).

O login é feito **por e-mail**, não por nome de usuário.

Documentação técnica

#	Documento	Conteúdo
01	Arquitetura	Apps, camadas, multi-tenancy, convenções do código
02	Ambiente local	Instalação, dados de demonstração, management commands
03	Configuração	Settings dev/prod, variáveis de ambiente, estáticos e anexos
04	Modelos de dados	Todos os models por app, enums, constraints, diagrama ER
05	Autorização	Motor de acessos: ações, perfis, escopos, <code>AuthorizationService</code>
06	Atividades e tarefas	Ciclos de vida, fila, prazos, sessões, processos
07	Contas e autenticação	Cadastro, confirmação de e-mail, login, senha
08	Notificações e auditoria	O que gera notificação, e-mail e registro de auditoria
09	Rotas	Referência de todas as URLs
10	Frontend	Templates, JS, CSS, context processors, template tags

#	Documento	Conteúdo
11	Testes	Como rodar, organização, helpers, teste JS
12	Deploy no Render	<code>render.yaml</code> , primeiro deploy, limitações do plano free
13	Pendências conhecidas	Defeitos e dívidas técnicas mapeados

Telas da aplicação

Capturas geradas automaticamente a partir da aplicação rodando localmente, com dados **fictícios** de demonstração (organização "Biasi", pessoas e clientes inventados). Resolução 1440×900. Geradas em 29/09/2026.

Acesso

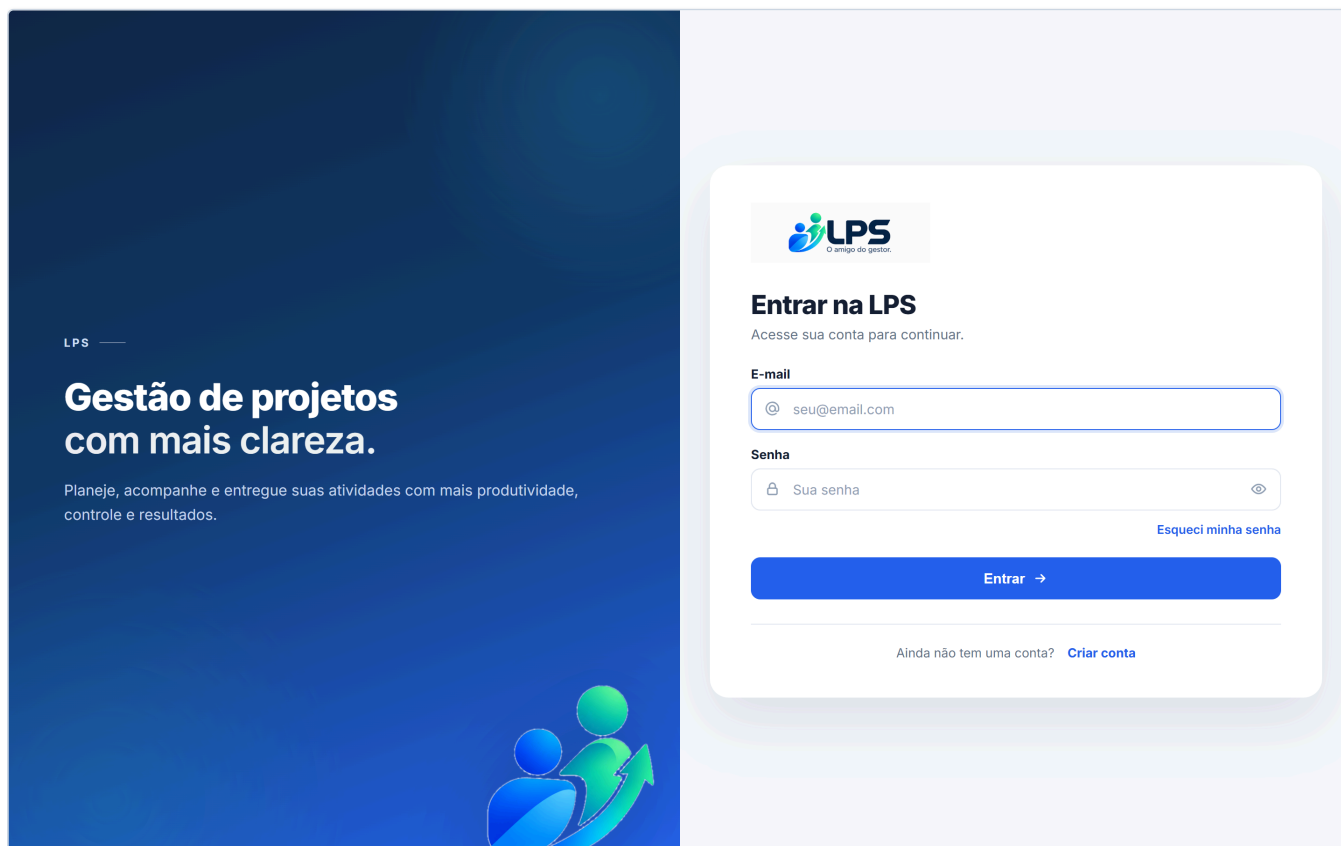


Figura 1 — Login por e-mail (/accounts/login/)

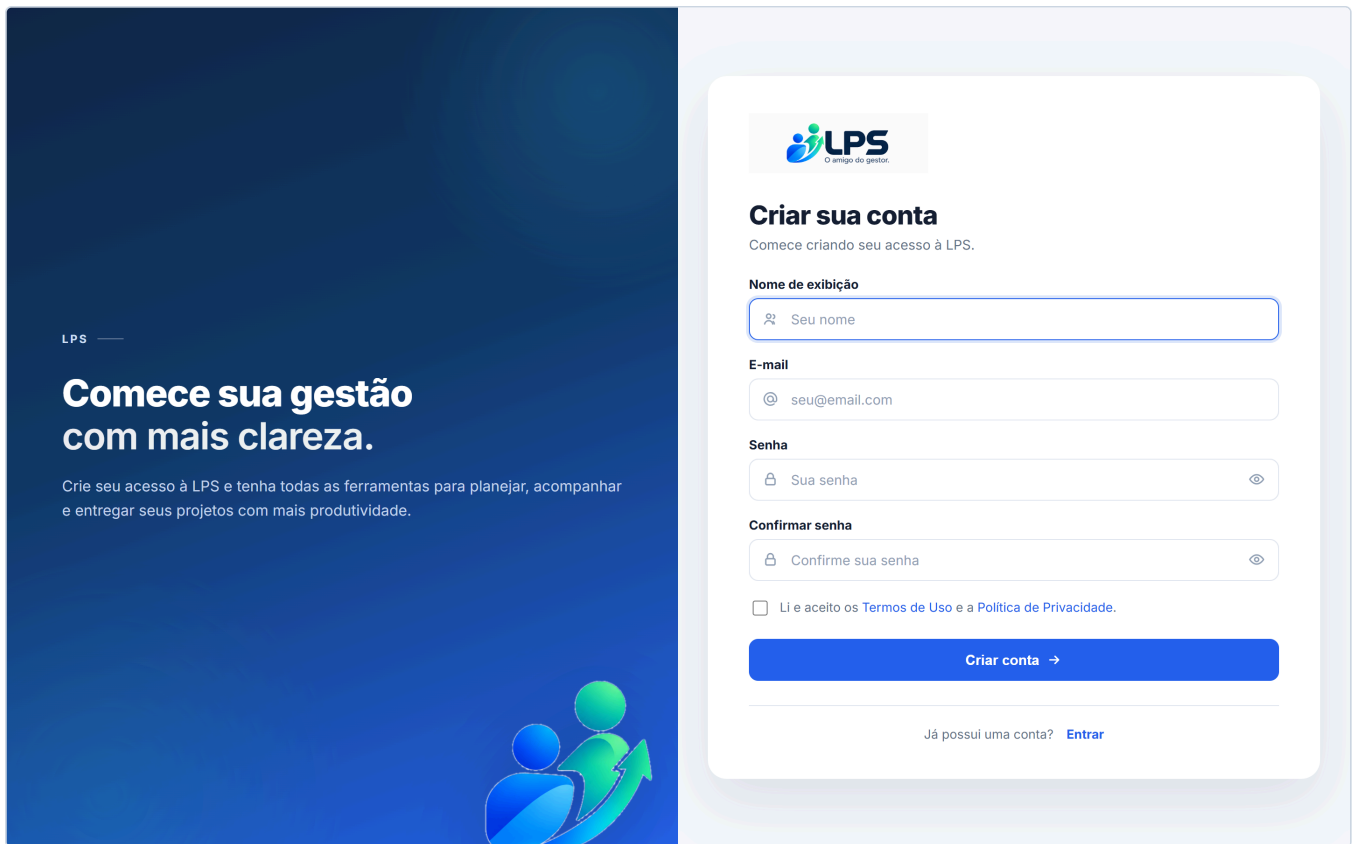


Figura 2 — Criar conta (/accounts/criar-conta/)

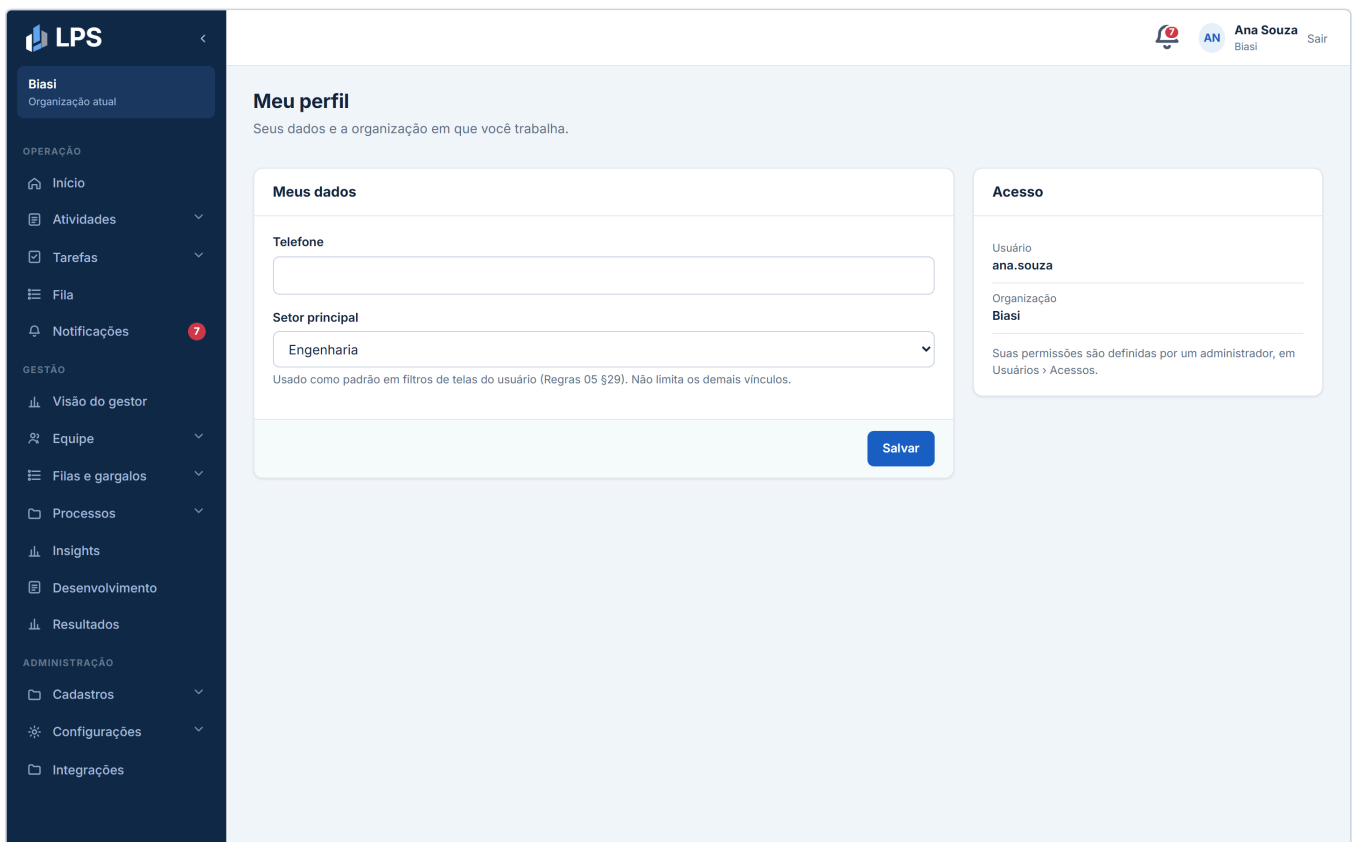


Figura 3 — Meu perfil (/accounts/me/)

Operação — atividades

LPS

Biasi
Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Fila

Notificações

7

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Configurações

Integrações

Olá, Ana Souza!

Aqui está o que precisa da sua atenção agora.

1
Minhas tarefas em aberto
Trabalho sob sua responsabilidade

0
Atrasadas
Passaram do prazo comprometido

0
Bloqueadas
Dependem de alguém ou de algo

1
Aguardando sua decisão
Prazos e conflitos para responder

Em execução agora

Nenhum cronômetro rodando

Quando você iniciar uma tarefa, o tempo aparece aqui e acompanha você em todas as telas.

Ver próximas tarefas

Próximas tarefas

Ver todas

TAREFA	SETOR	PRAZO	SITUAÇÃO
Diagrama unifilar revisado Projeto elétrico da Torre Norte aprovado	Engenharia	03/10/2026 21:35	Em fila
Conferir memorial de cálculo Revisão hidráulica da Ala B concluída	Engenharia	03/10/2026 21:35	Em fila

Preciso fazer alguma coisa?

Tarefa atribuída aguardando seu aceite

Dimensionar circuitos do 12º pavimento — atribuída por admin.demo em 29/09/2026 21:35

Aceitar

Analisar

Figura 4 — Início — o que precisa da minha atenção (/)

LPS

Biasi
Organização atual

OPERAÇÃO

Início

Atividades

Minhas

Participando

Do meu setor

Concluídas

Tarefas

Fila

Notificações

7

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Atividades

Os resultados que precisam acontecer.

Lista

Kanban

Calendário

Minhas

Do meu setor

Participando

Concluídas

Todas

Buscar atividade, cliente ou obr.

Atrasadas

Hoje

Bloqueadas

Status

Estagio

Setor

Ordenar

ATIVIDADE	CLIENTE / OBRA	SETOR	RESPONSÁVEL	TAREFAS	PRAZO	ESTÁGIO	ST.
Proposta comercial Shopping Vale Verde enviada	Shopping Vale Verde	Comercial	EL Elisa Prado	0 de 1	02/10/2026	Aguardando cliente	
Projeto elétrico da Torre Norte aprovado	Construtora Horizonte Torre Norte	Engenharia	AN Ana Souza	0 de 3	04/10/2026	A fazer	
Cotação de quadros de distribuição fechada	Construtora Horizonte Torre Norte	Compras	CA Carla Mendes	0 de 1	08/10/2026	Em análise	
Revisão hidráulica da Ala B concluída	Hospital São Lucas Expansão Ala B	Engenharia	BR Bruno Lima	0 de 1	11/10/2026	Revisão	

Mostrando 4 de 4 atividades

Figura 5 — Lista de atividades (/atividades/)

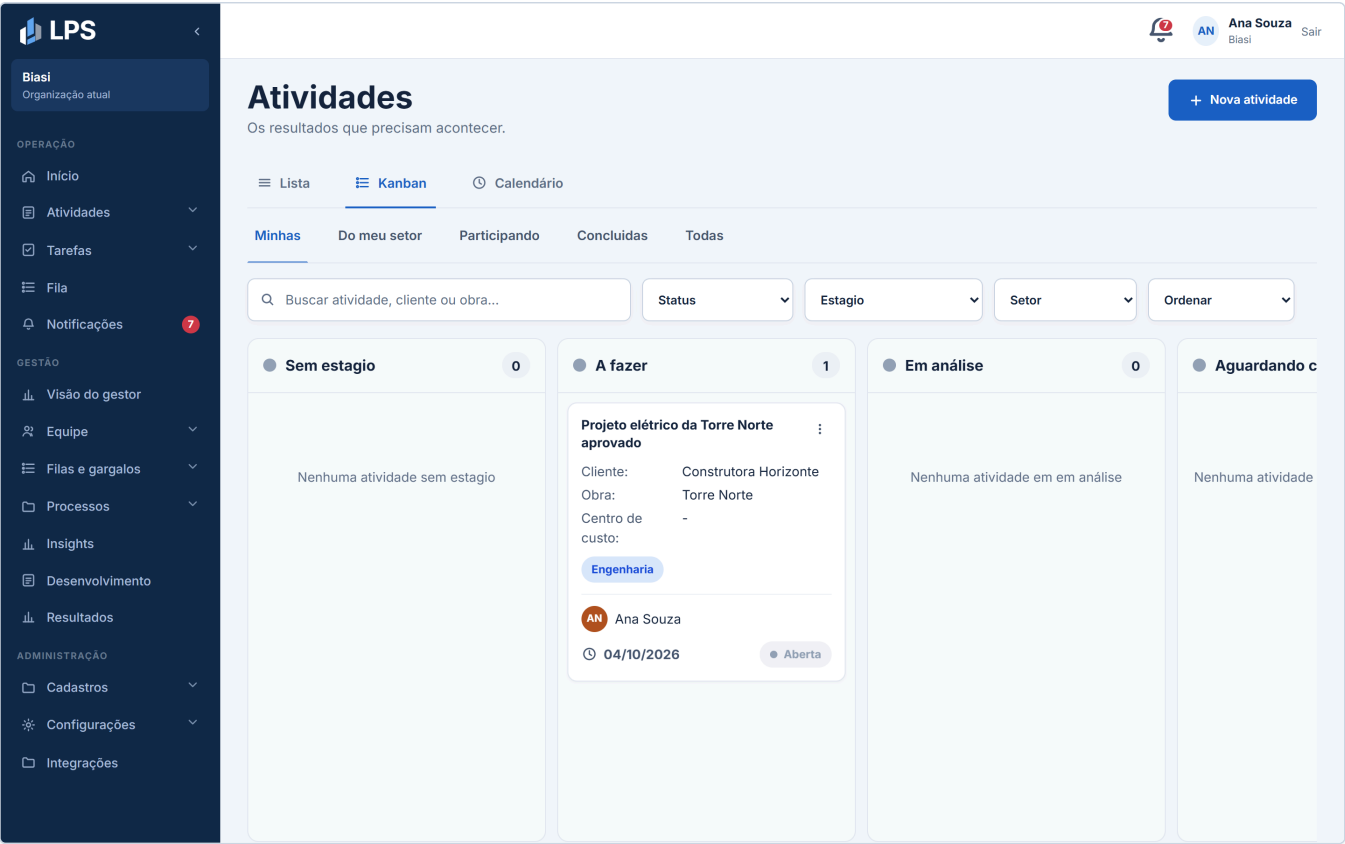


Figura 6 — Kanban de atividades por estágio (/atividades/kanban/)

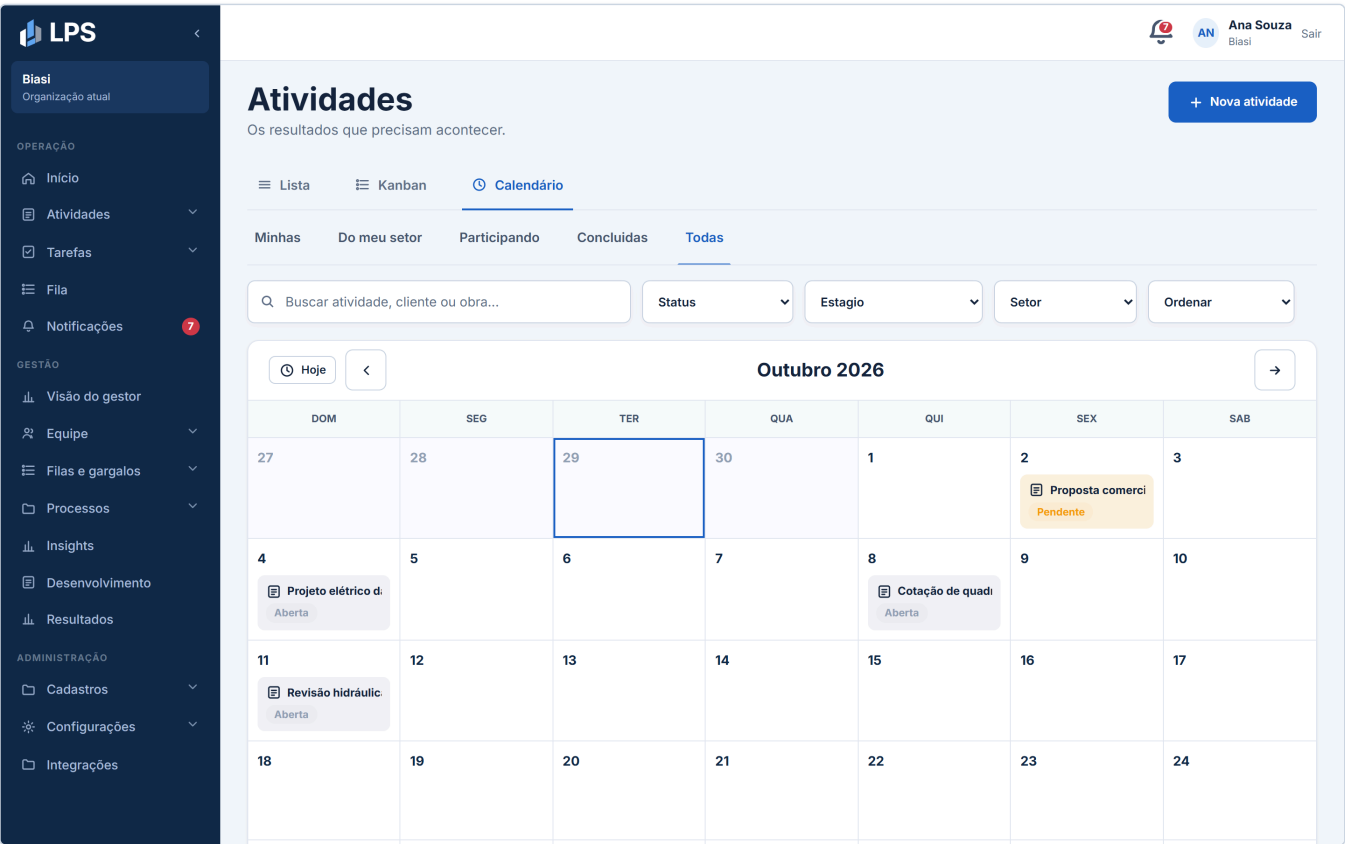


Figura 7 — Calendário de atividades por prazo (/atividades/calendario/)

Operação — tarefas e fila

LPS

Biasi
Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Minhas

Participando

Do meu setor

Concluídas

Fila

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Tarefas

Organize o trabalho do jeito que fizer mais sentido para o seu time.

+ Nova tarefa

Ver fila do setor

1
minhas tarefas

0
em execução

1
na fila

0
atrasadas

0
bloqueadas

Lista

Por atividade

Kanban

Calendário

Atribuídas a mim

Dos meus setores

Participando

Concluídas

Buscar tarefa, atividade, cliente ou obra...

Hoje

Atrasadas

Em fila

Em execução

Situação

Todas

Setor

Todos

Ordenar por

Prazo

Aplicar

TAREFA	ATIVIDADE	CLIENTE	OBRA	SETOR	RESPONSÁVEL	PRAZO	SITUAÇÃO
☐ Diagrama unifilar revisado	Projeto elétrico da Torre Norte aprovado	Construtora Horizonte	Torre Norte	Engenharia	ana.souza	03/10/2026 21:35	Em fila

Mostrando 1 de 1 tarefa

Figura 10 — Lista de tarefas (/tarefas/)

LPS

Biasi
Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Minhas

Participando

Do meu setor

Concluídas

Fila

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Tarefas

Organize o trabalho do jeito que fizer mais sentido para o seu time.

+ Nova tarefa

Ver fila do setor

Lista

Por atividade

Kanban

Calendário

Atribuídas a mim

Dos meus setores

Participando

Concluídas

Buscar por tarefa ou atividade...

Situação

Todas

Setor

Todos

Aplicar

Sem estágio

Backlog

Fazendo

Validação

Diagrama unifilar revisado

Atividade: Projeto elétrico da Torre Norte aprovado

Cliente: Construtora Horizonte

Obra: Torre Norte

Centro de custo: -

Em fila

Prazo: 03/10 21:35

+ Adicionar tarefa

+ Adicionar tarefa

+ Adicionar tarefa

+ Adicionar tarefa

+ Novo estágio

Adicione um est para organizar m suas tarefas

Figura 11 — Kanban de tarefas (/tarefas/kanban/)

LPS — Documentação técnica

10 / 55

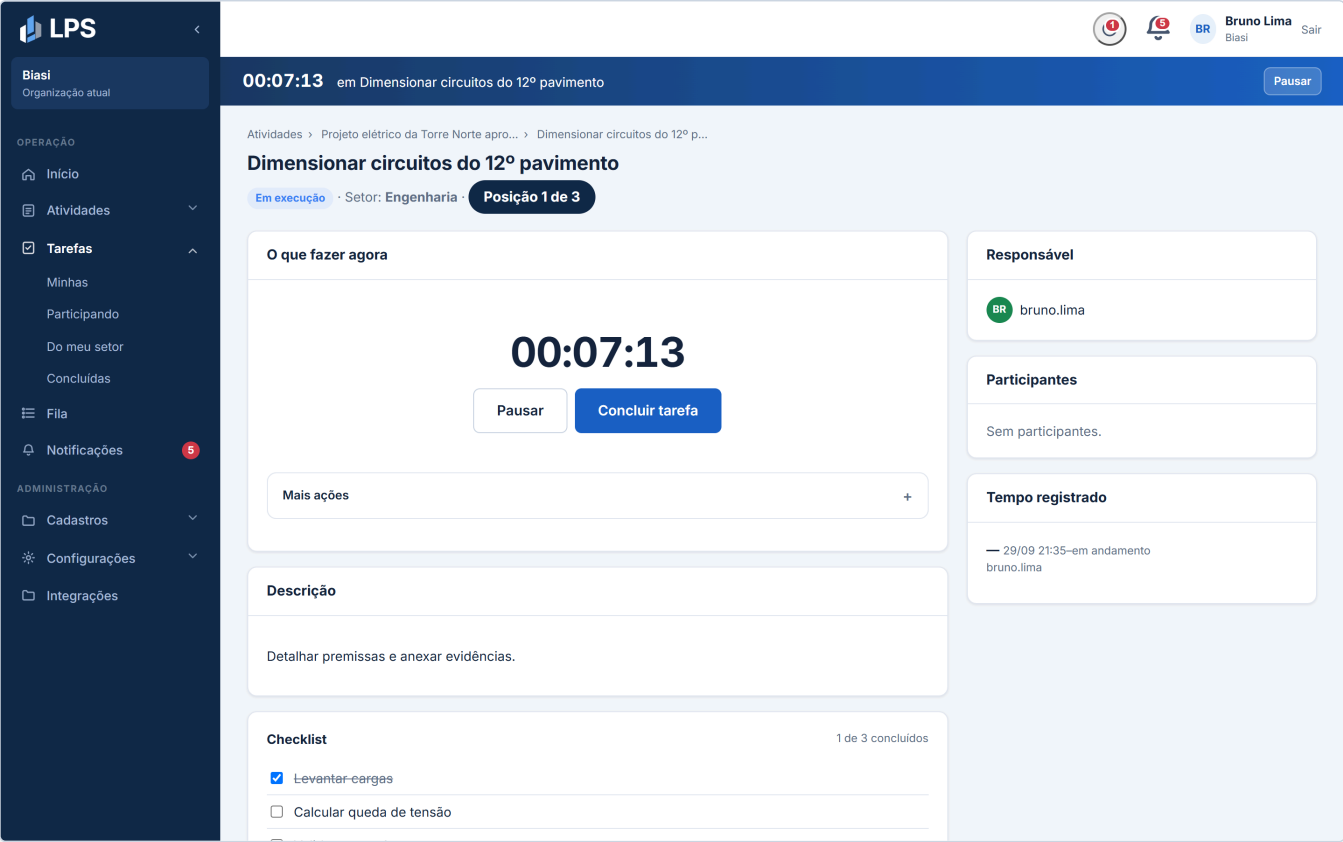


Figura 12 — Ficha da tarefa com sessão em andamento e checklist (/tarefas/<pk>/)

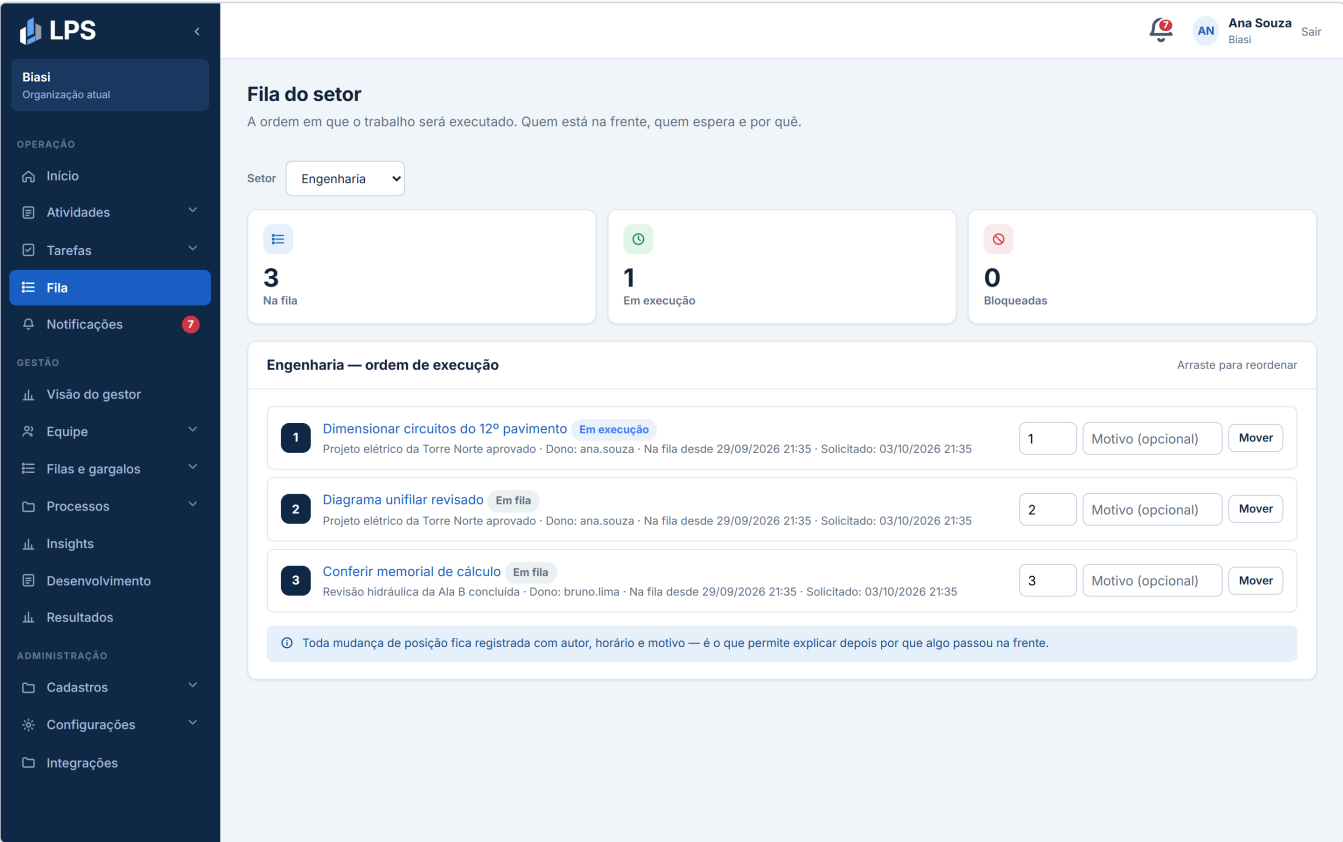


Figura 13 — Fila do setor (/fila/)

Acompanhamento

LPS

Biasi
Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Fila

Notificações7

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Configurações

Integrações

Notificações

O que precisa da sua atenção.

Ação necessária (5)

Não lidas (7)

Menções

Alertas (1)

Todas (7)

Ordenar por Mais recentes

INFORMATIVO Nova

Projeto elétrico da Torre Norte aprovado

bruno.lima: Consigo, falta só a carga do ar-condicionado.

Projeto elétrico da Torre Norte aprovado 00:35

bruno.lima: Consigo, falta só a carga do ar-condicionado.

ATENÇÃO Ação necessária

Levantar fornecedores de cabos

Esta tarefa está bloqueada e aguarda solução.

Compras Bloqueada hoje

A tarefa 'Levantar fornecedores de cabos' foi bloqueada: Aguardando especificação do cliente

Analisar

NOVA DELEGAÇÃO Ação necessária

Conferir memorial de cálculo

Admin Demo delegou esta atividade para você e aguarda sua ação.

Admin Demo delegou Engenharia Solicitado para 04/10 00:35

Abrir tarefa

NOVA DELEGAÇÃO Ação necessária

Diagrama unifilar revisado

Admin Demo delegou esta atividade para você e aguarda sua ação.

Admin Demo delegou Engenharia Solicitado para 04/10 00:35

Abrir tarefa

Figura 14 — Caixa de notificações (/notificacoes/)

LPS

Biasi
Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Fila

Notificações7

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Configurações

Integrações

Visão do gestor

O que precisa de decisão agora. Cada bloco abre o detalhe por trás do número.

Abrir histórico

Indicadores

Período (de)

dd/mm/aaaa

Período (até)

dd/mm/aaaa

Cliente

Todos

Responsável

Todos

Status

Todos

Centro de custo

Todos

Setor

Todos

Filtrar

5

Total de atividades

0

Em andamento

3

Pendentes

1

Concluídas (sucesso)

0

Declinadas
Atividades canceladas

0

Atrasadas

—

Tempo médio de conclusão

20,0%

Taxa de sucesso

0,0%

Taxa de declínio

0

Concluídas com pendências
Tiveram devolução no caminho

Filas por setor

Atividades por cliente

SETOR	NA FILA	AÇÕES
-------	---------	-------

CLIENTE	ATIVIDADES
---------	------------

Figura 15 — Gestão por exceção (/gestao/)

LPS — Documentação técnica

12 / 55



Biassi

Organização atual

OPERAÇÃO

 Início

 Atividades

 Tarefas

 Fila

 Notificações 7

GESTÃO

 Visão do gestor

 Equipe

 Filas e gargalos

 Processos

 Insights

 Desenvolvimento

 Resultados

ADMINISTRAÇÃO

 Cadastros

 Usuários

 Perfis e permissões

 Configurações

 Integrações

 AD Admin Demo Biassi Sair

Histórico

Tudo que aconteceu, em ordem, com autor e horário. O registro não é editável.

Todos

Tarefas

Prazos

Fila

Devoluções

Tempo

○ Admin Demo concluiu

ABERTA

→

CONCLUIDA

Motivo: Sucesso: Medição faturada e enviada ao cliente.

29/09/2026 21:35

● Admin Demo marcou como pendente: PENDENTE

ABERTA

→

PENDENTE

Motivo: Aguardando informações do cliente

29/09/2026 21:35

○ Fábio Nunes concluiu [Emitir nota fiscal](#)

EM_FILA

→

CONCLUIDA

29/09/2026 21:35

● Diego Rocha bloqueou a tarefa [Levantar fornecedores de cabos](#)

EM_FILA

→

BLOQUEADA

Motivo: Aguardando especificação do cliente

29/09/2026 21:35

○ Carla Mendes propôs um novo prazo [Consolidar três cotações](#)

2026-10-06 00:35:50.332793+00:00

29/09/2026 21:35

○ Bruno Lima iniciou a execução [Dimensionar circuitos do 12º pavimento](#)

EM_FILA

→

EM_EXECUCAO

29/09/2026 21:35

○ Admin Demo criou a tarefa Emitir nota fiscal [Emitir nota fiscal](#)

Emitir nota fiscal

29/09/2026 21:35

Administração

LPS

Blasi

Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Fila

Notificações

7

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Empresas

Setores

Clientes

Obras

Admin Demo

Sair

Blasi

Novo setor

Cadastros

O que existe na sua organização. Como a LPS se comporta fica em [Configurações](#).

Processos

Setores

Empresas

Obras

Centros de custo

Clientes

Tipos de atividade

Tipos de tarefa

Estágios de tarefa

Tags

Motivos

Q

Buscar pelo nome...

Buscar

NOME	DESCRIÇÃO	TAREFAS ABERTAS	SITUAÇÃO	AÇÕES
Comercial	—	1	● Ativo	Editar Inativar
Engenharia	—	3	● Ativo	Editar Inativar
Compras	—	2	● Ativo	Editar Inativar
Financeiro	—	0	● Ativo	Editar Inativar
Almoxarifado	—	0	● Ativo	Editar Inativar

Inativar preserva o histórico: o setor deixa de receber novas demandas, mas o que já passou por ele continua registrado.

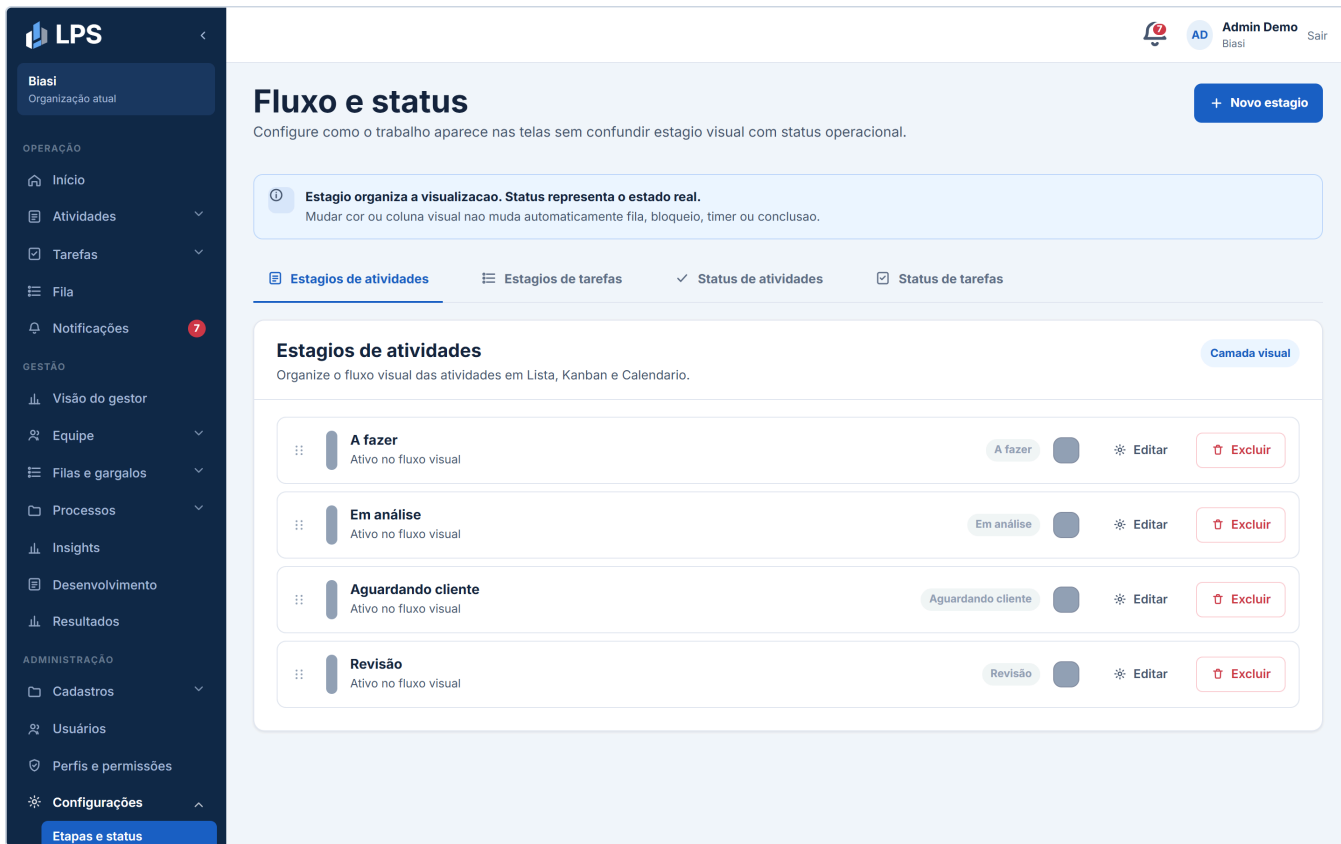


Figura 18 — Etapas e status (/painel/configuracoes/etapas-e-status/)

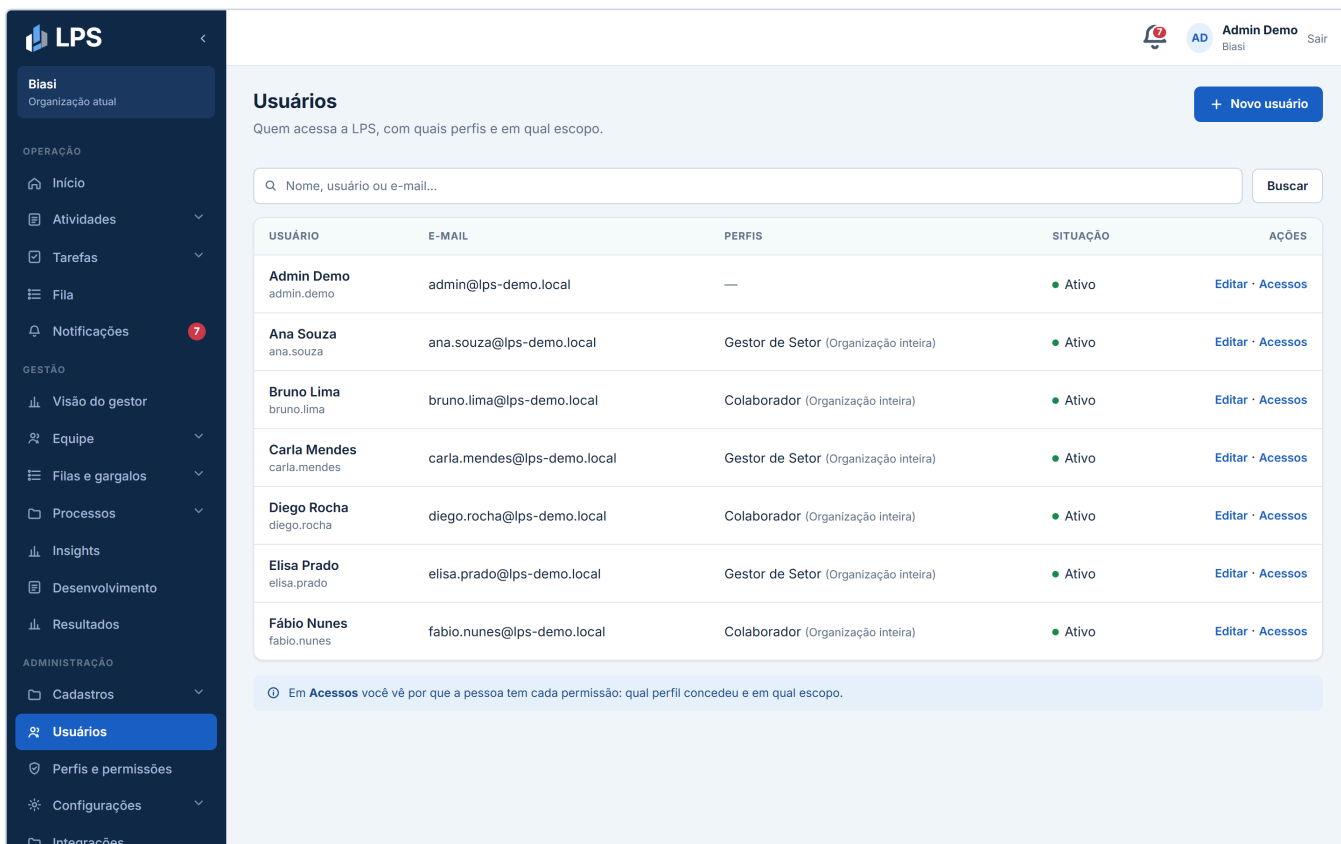


Figura 19 — Usuários da organização (/usuarios/)

LPS

Biasi

Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Fila

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Usuários > Novo usuário

Novo usuário

Identidade e onde a pessoa atua. O que ela pode fazer é definido em Acessos.

1

Dados do usuário

Identificação e situação.

Nome *

E-mail *

Usuário *

Organização *

Biasi

Senha *

A pessoa poderá alterá-la depois, em Perfil.

Confirmar senha *

☐ Almoxarifado

☐ Comercial

☐ Compras

☐ Engenharia

☐ Financeiro

☐ Almoxarifado

Setores em que atua

Setor não é permissão

"Setores em que atua" define onde a pessoa trabalha e para onde as tarefas do setor são notificadas — **não concede autorização**.

O que ela pode fazer é definido em Acessos, após salvar, combinando perfil e escopo.

Figura 20 — Cadastro de usuário pelo administrador (/usuarios/novo/)

LPS

Biasi

Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Fila

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

Usuários > ana.souza > Acessos

Acessos de Ana Souza

Quem pode o quê, onde, e por qual origem.

Perfis atribuídos

Cada atribuição vale dentro de um escopo

PERFIL	ONDE VALE	AÇÕES
Gestor de Setor	Organização inteira	Remover

Atribuir perfil

Perfil *

Onde vale *

Organização inteira

Empresa

Setor

Obra

Centro de custo

Relação

Acesso efetivo

O que esta pessoa pode fazer hoje, e de onde vem cada permissão.

Aprovar pendência da atividade

Organização inteira · via Gestor de Setor

Assumir atividade do grupo

Organização inteira · via Gestor de Setor

Criar atividade

Organização inteira · via Gestor de Setor

Editar atividade

Organização inteira · via Gestor de Setor

Marcar atividade como pendente

Organização inteira · via Gestor de Setor

Visualizar atividades

Organização inteira · via Gestor de Setor

Visualizar todas as atividades

Organização inteira · via Gestor de Setor

Aceitar atribuição

Organização inteira · via Gestor de Setor

Assumir tarefa

Organização inteira · via Gestor de Setor

Atribuir participante

Figura 21 — Acessos de uma pessoa, com a origem (/usuarios/<pk>/acessos/)

LPS — Documentação técnica

15 / 55

LPS

Biasi

Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Fila

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Usuários

Perfis e permissões

Configurações

Integrações

AD

Admin Demo

Biasi

Sair

Perfis e permissões

+ Novo perfil

Um perfil responde: o que esta pessoa pode fazer? Onde ela pode fazer é definido ao atribuir o perfil, na tela de acessos do usuário.

Q

Buscar ação. Ex.: fila

Mostrar

Todas

Filtrar

Colaborador

Editar dados

1

Atividades

☐

Alterar dono da atividade

sensível

Permite transferir a responsabilidade pelo resultado para outra pessoa. atividade.alterar_dono

☐

Aprovar pendência da atividade

sensível

Permite decidir uma pendência que aguarda aprovação do gestor, devolvendo a atividade para quem a designou. atividade.aprovar_pendencia

☒

Assumir atividade do grupo

Permite se tornar dono de uma atividade endereçada ao setor da pessoa, direto na fila do grupo. atividade.assumir

☐

Cancelar atividade

sensível

Permite cancelar a atividade registrando o motivo. atividade.cancelar

☐

Concluir atividade

Permite encerrar a atividade quando o resultado foi alcançado. atividade.concluir

☒

Criar atividade

Permite registrar um novo resultado a ser alcançado. atividade.criar

☐

Editar atividade

Permite alterar título, descrição, prazo e contexto da atividade. atividade.editar

☒

Marcar atividade como pendente

Permite pausar a atividade registrando o motivo da pendência, com comentário obrigatório. atividade.marcar_pendente

☐

Reabrir atividade

sensível

Permite reabrir uma atividade já concluída. atividade.reabrir

☒

Visualizar atividades

Perfis

Colaborador

3 atribuições

Gestor de Setor

3 atribuições

Administrador

0 atribuições

Quem possui este perfil

bruno.lima

Organização inteira

diego.rocha

Organização inteira

fabio.nunes

Organização inteira

Figura 22 — Matriz de permissões (/permissões/)

LPS

Biasi

Organização atual

OPERAÇÃO

Início

Atividades

Tarefas

Fila

Notificações

GESTÃO

Visão do gestor

Equipe

Filas e gargalos

Processos

Insights

Desenvolvimento

Resultados

ADMINISTRAÇÃO

Cadastros

Empresas

Setores

Clientes

Obras

AD

Admin Demo

Biasi

Sair

Processos

+ Novo processo

Gerencie os modelos reutilizáveis da organização.

0

Processos ativos

0

Em rascunho

0

Tipos de atividade

Q

Buscar pelo nome...

Todos os tipos de atividade

Todos os status

Filtrar

NOME	TIPO DE ATIVIDADE	INPUTS	OUTPUT	FLUXO	VERSÃO	STATUS	AÇÕES
Nenhum processo cadastrado Um processo descreve como uma categoria recorrente de trabalho normalmente deve acontecer — com inputs, output, critérios de aceite e fluxo padrão. Criar primeiro processo							

Processo não é obrigatório: atividades ad hoc continuam podendo ser criadas apenas com título, dono e prazo.

Figura 23 — Processos (/processos/)

LPS — Documentação técnica

16 / 55

01 — Arquitetura

Visão geral de como o código da LPS está organizado: quais apps existem, o que cada um faz, como as camadas se relacionam e quais convenções o código segue. As regras de negócio que motivam cada decisão estão em `Regras/`; os docstrings do código citam essas seções (ex.: "Regras 05 §43").

1. Apps

O projeto Django fica em `config/` (settings, urls, wsgi/asgi). Os apps de negócio estão na raiz:

App	Responsabilidade	Principais arquivos
<code>core</code>	Tenant (<code>Organization</code>) e cadastros da organização: empresas, setores, obras, clientes, centros de custo, tags, estágios de Kanban, cores e rótulos de status. Também hospeda as telas de usuários e de segurança e os mixins base.	<code>models.py</code> , <code>services.py</code> , <code>mixins.py</code> , <code>colors.py</code> , <code>sanitize.py</code> , <code>widgets.py</code>
<code>accounts</code>	Identidade: <code>Profile</code> (1:1 com <code>User</code> , liga a pessoa a uma organização), participação em setores (<code>UserSector</code>), cadastro com confirmação de e-mail, login por e-mail.	<code>models.py</code> , <code>auth_backends.py</code> , <code>forms.py</code> , <code>signals.py</code> , <code>views.py</code>
<code>acessos</code>	Motor de autorização: catálogo de ações, perfis de acesso, escopos, concessões. Responde "quem pode fazer o quê, onde e por quê". Não tem telas próprias (ficam em <code>core</code>).	<code>catalog.py</code> , <code>models.py</code> , <code>services.py</code> , <code>testing.py</code>
<code>activities</code>	Núcleo operacional: atividades, tarefas, fila por setor, negociação de prazo, bloqueios, devoluções, sessões de trabalho, checklist, anexos, mensagens com @menções.	<code>models.py</code> , <code>services.py</code> , <code>views.py</code> , <code>templatetags/lps.py</code>
<code>processes</code>	Modelos de processo reutilizáveis e versionados (inputs, output, critérios de aceite, fluxo por setor).	<code>models.py</code> , <code>services.py</code> , <code>views.py</code>
<code>notifications</code>	Notificações in-app, caixa de entrada com categorias, e-mails síncronos.	<code>models.py</code> , <code>services.py</code> , <code>recipients.py</code> , <code>views.py</code>
<code>audit</code>	<code>AuditLog</code> : trilha de auditoria somente-inclusão de eventos de negócio e de segurança.	<code>models.py</code> , <code>services.py</code>
<code>painel</code>	Telas "em construção" que mantêm o menu lateral fiel a Telas/ <code>MENU_E_SUBMENUS_LPS.md</code> , mais duas telas de configuração emprestadas de <code>core</code> .	<code>views.py</code> , <code>urls.py</code>

As pastas `demands/` e `workflows/` são restos de uma versão anterior: não estão em `INSTALLED_APPS` e não têm código (ver [13_PENDENCIAS_CONHECIDAS.md](#)).

Dependências entre apps

Levantado a partir dos imports (fora de testes e migrations) e das FKs entre apps:

App	Usa
<code>audit</code>	nenhum app (só FKs para <code>activities</code> e <code>User</code>)
<code>accounts</code>	<code>acessos</code> (invalidação de cache); FKs para <code>core</code>
<code>painel</code>	<code>core</code>
<code>processes</code>	<code>acessos</code> , <code>core</code>
<code>notifications</code>	<code>accounts</code> ; FKs para <code>activities</code>
<code>acessos</code>	<code>accounts</code> , <code>activities</code> , <code>audit</code> , <code>core</code>
<code>core</code>	<code>accounts</code> , <code>acessos</code> , <code>activities</code> , <code>audit</code>
<code>activities</code>	<code>acessos</code> , <code>audit</code> , <code>core</code> , <code>notifications</code> ; FK para <code>processes</code>

Há dependências circulares (`core` ↔ `acessos` ↔ `activities`). Elas são resolvidas com imports **dentro das funções** e FKs por string (`"core.Organization"`). Ao mover código entre apps, mantenha esse padrão para não criar erro de import circular na inicialização.

2. Camadas

URL → View (fina) → Service (regra + autorização + auditoria + notificação) → Model

- **Views** resolvem a organização, carregam objetos já filtrados por ela, validam o formulário e chamam um serviço. Views de ação herdam de `ServiceActionView` (em `activities/views.py`): chamam o serviço, convertem o erro em mensagem (`django.contrib.messages`) e redirecionam.
- **Services** (`*/services.py`) concentram a regra de negócio. Cada método público: 1. verifica a autorização (`require_action(user, catalog.X, recurso)`); 2. valida a transição de estado; 3. grava; 4. registra em `AuditService.log(...)`; 5. dispara `NotificationService.notify(...)` e, quando aplicável, e-mail.
- **Models** guardam dados e propriedades de exibição; quase não têm regra.

Erros de negócio são exceções específicas por app, sempre com mensagem pronta para o usuário:

Exceção	Onde
<code>ActivityError</code>	<code>activities/services.py</code>
<code>CadastroError</code>	<code>core/services.py</code>
<code>ProcessError</code>	<code>processes/services.py</code>
<code>AuthorizationError</code>	<code>acessos/services.py</code> (convertida em <code>ActivityError</code> por <code>require_action</code>)

3. Multi-tenancy

`core.Organization` é o tenant. Tudo pertence a uma organização:

1. Uma pessoa pertence a **uma** organização via `accounts.Profile.organization` (Regras 05 §7). O `Profile` é criado vazio por signal quando o `User` é criado; quem define a organização é um administrador (tela `/usuarios/`).
2. Cadastros e registros de acesso têm FK `organization`. `Activity` tem FK direta; `Task` herda pela atividade; `QueueEntry`, pelo setor.
3. `core.mixins.OrganizationRequiredMixin` resolve `self.organization` em toda view da LPS. Sem organização, redireciona para `/accounts/me/` com a mensagem "Seu usuário ainda não está vinculado a uma organização".
4. As views filtram tudo por `organization=self.organization` (ou `activity__organization=...`); objetos de outra organização dão 404.
5. O motor de autorização aplica um **teto de tenant**: se o recurso é de outra organização, nenhuma concessão vale (doc 05 §43). Ver [05_AUTORIZACAO.md](#).

O Admin do Django (`/admin/`) **não** é filtrado por organização — é a ferramenta do operador da plataforma.

4. Convenções do código

- **Autorização nega por padrão.** `ActionRequiredMixin` exige o atributo `required_action`; sem ele, levanta `ImproperlyConfigured`.
- **Esconder no menu é UX, não segurança.** O context processor `acessos.context_processors.navigation` só decide o que aparece; a verificação real acontece na view ou no serviço (doc 09 §11).
- **Auditoria só por `AuditService.log`.** Nenhum código cria `AuditLog` diretamente.
- **HTML de usuário só entra por `core/sanitize.py`** (`sanitize_description`, baseado em `nh3`, com lista fixa de tags permitidas).
- **Nada é apagado de verdade** quando há histórico: cadastros usam `is_active`, vínculos usam `removed_at` (`UserSector`, `TaskExecutor`), concessões de acesso usam `is_active=False`. Mudanças de dono/responsável geram linhas de log (`OwnerChangeLog`, `TaskResponsavelChangeLog`).
- **Nomes duplicados** em cadastros são checados sem diferenciar maiúsculas e espaços (`core.services._assert_unique_name`, Regras 05 §22-23).
- **Status × estágio.** `status` é o estado de negócio (fixo no código); `stage` é a coluna visual do Kanban (configurável por organização). Mover um card de estágio nunca altera o status.

- **Cores e rótulos** de status/urgência são configuráveis por organização (`core.EnumColor`) mas o **código** do enum nunca muda de significado; a resolução passa por `core.colors.EnumColorResolver`.
- **Idioma**. Interface, docstrings e mensagens em português; nomes de classes e funções em inglês; valores de enum em português maiúsculo (`EM_FILA` , `CONCLUIDA`).

02 — Ambiente local

Como colocar a LPS para rodar na sua máquina, do zero, com dados de demonstração e um usuário capaz de usar todas as telas.

1. Pré-requisitos

- Python 3.12 ou superior (o time usa 3.14 localmente; o Render usa 3.12.7 — ver [12_DEPLOY_RENDER.md](#)).
- Git.
- Node 18+ **apenas** se for rodar o teste JavaScript do checklist ([11_TESTES.md](#)).

Os comandos abaixo são para PowerShell no Windows. Em Linux/macOS, troque `.venv\Scripts\Activate.ps1` por `source .venv/bin/activate` e `copy` por `cp`.

2. Instalação

```
git clone https://github.com/Lucasrafael703/workflow.git
cd workflow
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
```

Abra o `.env` e defina ao menos `SECRET_KEY` (qualquer string longa e aleatória serve em dev). O resto pode ficar como está: em dev o e-mail sai no console e o banco é SQLite (`db.sqlite3`, ignorado pelo Git). Todas as variáveis estão em [03_CONFIGURACAO.md](#).

Use `.venv/`, não `venv/`. A pasta `venv/` que existe no repositório local é um ambiente quebrado de outra máquina (ver [13_PENDENCIAS_CONHECIDAS.md](#)).

3. Banco e dados de demonstração

```
python manage.py migrate
python manage.py seed_lps_demo
```

`seed_lps_demo` é idempotente (pode rodar várias vezes) e:

1. roda `seed_acoes`, que grava o catálogo de ações de `acessos/catalog.py`;
2. cria a organização **Biasi** e a empresa **Biasi Engenharia** (mude com `--org-name` e `--company-name`);
3. cria os setores Comercial, Engenharia, Compras, Financeiro e Almoxarifado;
4. cria os perfis de acesso sugeridos — **Colaborador**, **Gestor de Setor** e **Administrador** — com suas ações.

Ele **não** cria usuários.

4. Primeiro usuário

```
python manage.py createsuperuser
```

Preencha o **e-mail**: o login da LPS é por e-mail (`accounts.auth.backends.EmailBackend`), então um superusuário sem e-mail não consegue entrar pela tela de login.

Em seguida, **vincule o usuário a uma organização**. Sem isso, toda tela da LPS redireciona para `/accounts/me/` com a mensagem "Seu usuário ainda não está vinculado a uma organização":

1. `python manage.py runserver` e entre em `http://127.0.0.1:8000/admin/`;
2. seção **Accounts** → **Perfis** (não confundir com "Perfis de acesso", do app Acessos) → seu usuário → campo **organização** = Biasi → Salvar.

Ou, pelo shell:

```
python manage.py shell -c "from django.contrib.auth import get_user_model; from core.models import Organization; u = get_user_model().objects.get(username='SEU_USUARIO'); u.profile.organization = Organization.objects.get(name='Biasi'); u.profile.save()"
```

Um superusuário ativo passa por todas as verificações de autorização (`AuthorizationService.is_platform_admin`), então não precisa de perfil de acesso. Para testar como um usuário comum, crie outra pessoa em `/usuarios/novo/` e atribua um perfil em `/usuarios/<id>/acessos/`.

5. Rodando

```
python manage.py runserver
```

Ou dê dois cliques em `iniciar_lps.bat`: ele sobe o servidor em `127.0.0.1:8000` numa janela minimizada (usando `.venv`) e abre o Chrome.

`manage.py` usa `config.settings.dev` por padrão (DEBUG ligado, SQLite, e-mail no console).

6. Management commands

Comando	App	O que faz	Quando usar
<code>seed_acoes</code>	<code>acessos</code>	Sincroniza o catálogo de <code>acessos/catalog.py</code> com o banco (<code>update_or_create</code>). Ações que saíram do catálogo ficam <code>is_active=False</code> ; nunca são apagadas.	Sempre que <code>catalog.py</code> mudar. Já é chamado por <code>seed_lps_demo</code> .
<code>seed_lps_demo</code>	<code>core</code>	Organização, empresa, setores e perfis sugeridos (ver seção 3). Opções <code>-org-name</code> , <code>--company-name</code> .	Primeiro setup; ambientes de demonstração.
<code>ensure_superuser</code>	<code>accounts</code>	Cria um superusuário a partir de <code>DJANGO_SUPERUSER_USERNAME</code> , <code>DJANGO_SUPERUSER_EMAIL</code> e <code>DJANGO_SUPERUSER_PASSWORD</code> , só se esse username ainda não existir. Sem as variáveis, não faz nada.	Deploy automatizado (roda no build do Render).
<code>check_overdue_tasks</code>	<code>activities</code>	Marca tarefas com prazo comprometido vencido, notifica (in-app + e-mail) uma única vez por tarefa.	Deve ser agendado (Agendador de Tarefas do Windows, cron). Não há worker assíncrono.

7. Problemas comuns

Sintoma	Causa	Solução
<code>ImproperlyConfigured: Set the SECRET_KEY environment variable</code>	<code>.env</code> ausente ou sem <code>SECRET_KEY</code>	Copie <code>.env.example</code> para <code>.env</code> e preencha.
Toda tela volta para "Meu perfil" com aviso de organização	<code>Profile.organization</code> vazio	Seção 4.
Login diz que e-mail/senha estão errados para o superusuário	Superusuário criado sem e-mail	Defina o e-mail no Admin ou recrie.

Sintoma	Causa	Solução
Conta criada em "Criar conta" não entra	A conta fica inativa até confirmar o código de e-mail; o código aparece no console do <code>runserver</code> em dev	Ver 07_CONTAS_E_AUTENTICACAO.md .
Botões/menus não aparecem para um usuário comum	Falta perfil de acesso com a ação no escopo certo	Ver 05_AUTORIZACAO.md .
<code>InconsistentMigrationHistory</code> depois de um <code>git pull</code>	Banco local criado com migrations antigas que foram reescritas	Em dev, apague <code>db.sqlite3</code> e rode <code>migrate + seed_lps_demo</code> de novo.

03 — Configuração

Como os settings estão divididos, todas as variáveis de ambiente que o código lê, e como arquivos estáticos e anexos são guardados e servidos.

1. Settings

```
config/settings/
├── base.py # comum: apps, middleware, templates, auth, i18n, estáticos, e-mail
├── dev.py  # DEBUG=True, SQLite, e-mail no console
└── prod.py # DEBUG=False, Postgres via DATABASE_URL, whitenoise, HTTPS
```

`base.py` lê o arquivo `.env` na raiz do projeto (`environ.Env.read_env`). Variáveis já definidas no ambiente têm prioridade sobre o `.env`.

Qual settings é usado depende de **como** o Django é iniciado:

Entrada	Default de <code>DJANGO_SETTINGS_MODULE</code>
<code>manage.py</code>	<code>config.settings.dev</code>
<code>config/wsgi.py</code> (gunicorn)	<code>config.settings.prod</code>
<code>config/asgi.py</code>	<code>config.settings.prod</code>

Por isso, no Render, `DJANGO_SETTINGS_MODULE=config.settings.prod` precisa estar definido: o build roda `manage.py` (`collectstatic`, `migrate`), que sozinho usaria `dev`.

2. Variáveis de ambiente

Variável	Default	Lida em	Observação
<code>SECRET_KEY</code>	— obrigatória	<code>base.py</code>	Sem ela o Django não inicia.
<code>ALLOWED_HOSTS</code>	<code>[]</code>	<code>base.py</code>	Lista separada por vírgula. Em dev com <code>DEBUG=True</code> , <code>localhost</code> já é aceito.
<code>ACTIVITY_FILES_ROOT</code>	<code><projeto>/atividade_arquivos</code>	<code>base.py</code>	Pasta dos anexos de atividade (seção 4).
<code>EMAIL_HOST</code>	<code>" "</code>	<code>base.py</code>	
<code>EMAIL_PORT</code>	<code>587</code>	<code>base.py</code>	
<code>EMAIL_HOST_USER</code>	<code>" "</code>	<code>base.py</code>	
<code>EMAIL_HOST_PASSWORD</code>	<code>" "</code>	<code>base.py</code>	
<code>EMAIL_USE_TLS</code>	<code>True</code>	<code>base.py</code>	
<code>DEFAULT_FROM_EMAIL</code>	<code>no-reply@example.com</code>	<code>base.py</code>	Remetente dos e-mails.
<code>ADMIN_GROUP_NAME</code>	<code>ADMIN</code>	<code>base.py</code>	Lida, mas não usada por nenhum código hoje.
<code>EMAIL_BACKEND</code>	<code>console (dev) · SMTP (prod)</code>	<code>dev.py</code> , <code>prod.py</code>	
<code>DATABASE_URL</code>	— obrigatória em prod	<code>prod.py</code>	Ex.: <code>postgres://user:senha@host:5432/banco</code> .
<code>SECURE_SSL_REDIRECT</code>	<code>True</code>	<code>prod.py</code>	Redireciona HTTP → HTTPS.
<code>SECURE_HSTS_SECONDS</code>	<code>3600</code>	<code>prod.py</code>	
<code>CSRF_TRUSTED_ORIGINS</code>	<code>[]</code>	<code>prod.py</code>	Com esquema: <code>https://*.onrender.com</code> .

Variável	Default	Lida em	Observação
DJANGO_SUPERUSER_USERNAME	—	ensure_superuser	
DJANGO_SUPERUSER_EMAIL	" "	ensure_superuser	Preencha: o login é por e-mail.
DJANGO_SUPERUSER_PASSWORD	—	ensure_superuser	
DJANGO_SETTINGS_MODULE	ver seção 1	Django	

Todas estão listadas (as opcionais comentadas) em `.env.example`.

3. Outras configurações relevantes

Setting	Valor	Por quê
LANGUAGE_CODE / TIME_ZONE	pt-br / America/Sao_Paulo	
USE_TZ	True	Datas gravadas em UTC, exibidas no fuso local.
AUTHENTICATION_BACKENDS	accounts.auth_backends.EmailBackend	Login por e-mail (Telas/09_01_LOGIN.md). Único backend: ModelBackend por username não está ativo.
LOGIN_URL / LOGIN_REDIRECT_URL / LOGOUT_REDIRECT_URL	login / home / login	
DEFAULT_AUTO_FIELD	BigAutoField	
Context processors próprios	notifications...unread_notifications_count, acessos...navigation, activities...my_active_sessions	Ver 10_FRONTEND.md.
SECURE_PROXY_SSL_HEADER (prod)	("HTTP_X_FORWARDED_PROTO", "https")	O Render termina o TLS no proxy. Sem isso, SECURE_SSL_REDIRECT entra em loop de redirecionamento.
Cookies (prod)	SESSION_COOKIE_SECURE, CSRF_COOKIE_SECURE	
HSTS (prod)	INCLUDE_SUBDOMAINS e PRELOAD ligados	

4. Arquivos estáticos, mídia e anexos

Tipo	Pasta de origem	URL	Quem serve
Estáticos do projeto	static/ (STATICFILES_DIRS)	/static/	dev: runserver · prod: whitenoise a partir de staticfiles/
Estáticos coletados	staticfiles/ (STATIC_ROOT, gerado por collectstatic, ignorado pelo Git)	/static/	whitenoise
Mídia	media/ (MEDIA_ROOT)	/media/	django.views.static.serve
Anexos de atividade	ACTIVITY_FILES_ROOT	/atividade-arquivos/	django.views.static.serve

- Em prod, STORAGES["staticfiles"] usa whitenoise.storage.CompressedManifestStaticFilesStorage (nomes com hash e compressão). Por isso é preciso rodar collectstatic no build. Em dev o storage padrão é mantido para não exigir collectstatic.
- Anexos são gravados pelo storage activity_files_storage() (activities/models.py) no caminho <empresa>/<código da atividade>/<arquivo> (Regras 12 e 13); sem empresa, vai para sem-empresa/.

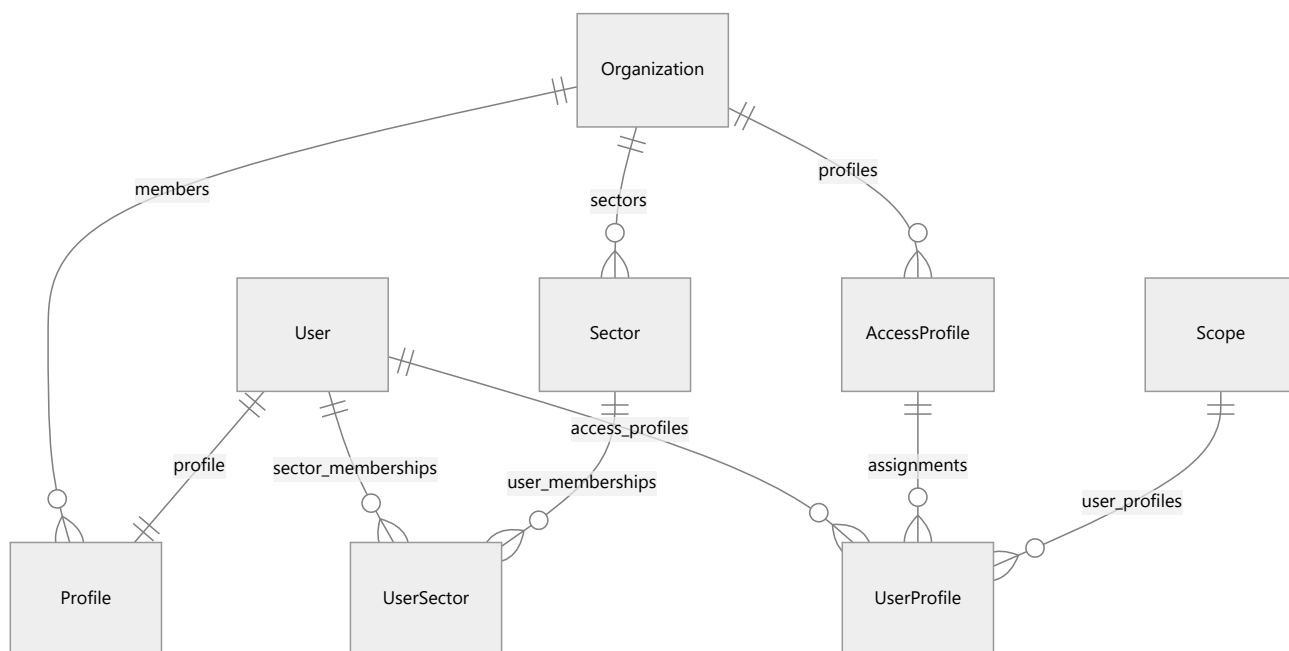
- `/media/` e `/atividade-arquivos/` são servidos pelo próprio Django **em todos os ambientes** (`config/urls.py`), porque no Render não há nginx na frente. Essas rotas **não exigem login** — ver [13_PENDENCIAS_CONHECIDAS.md](#).
- No plano free do Render o disco não é persistente: anexos enviados se perdem a cada deploy ou reinício.

04 — Modelos de dados

Todos os modelos da aplicação, agrupados por app, com os campos que importam, relacionamentos (`related_name`), enums e constraints. Campos triviais (`created_at` , `name`) aparecem só quando têm regra. A modelagem conceitual do produto está em `Regras/08_BANCO_DE_DADOS.md`.

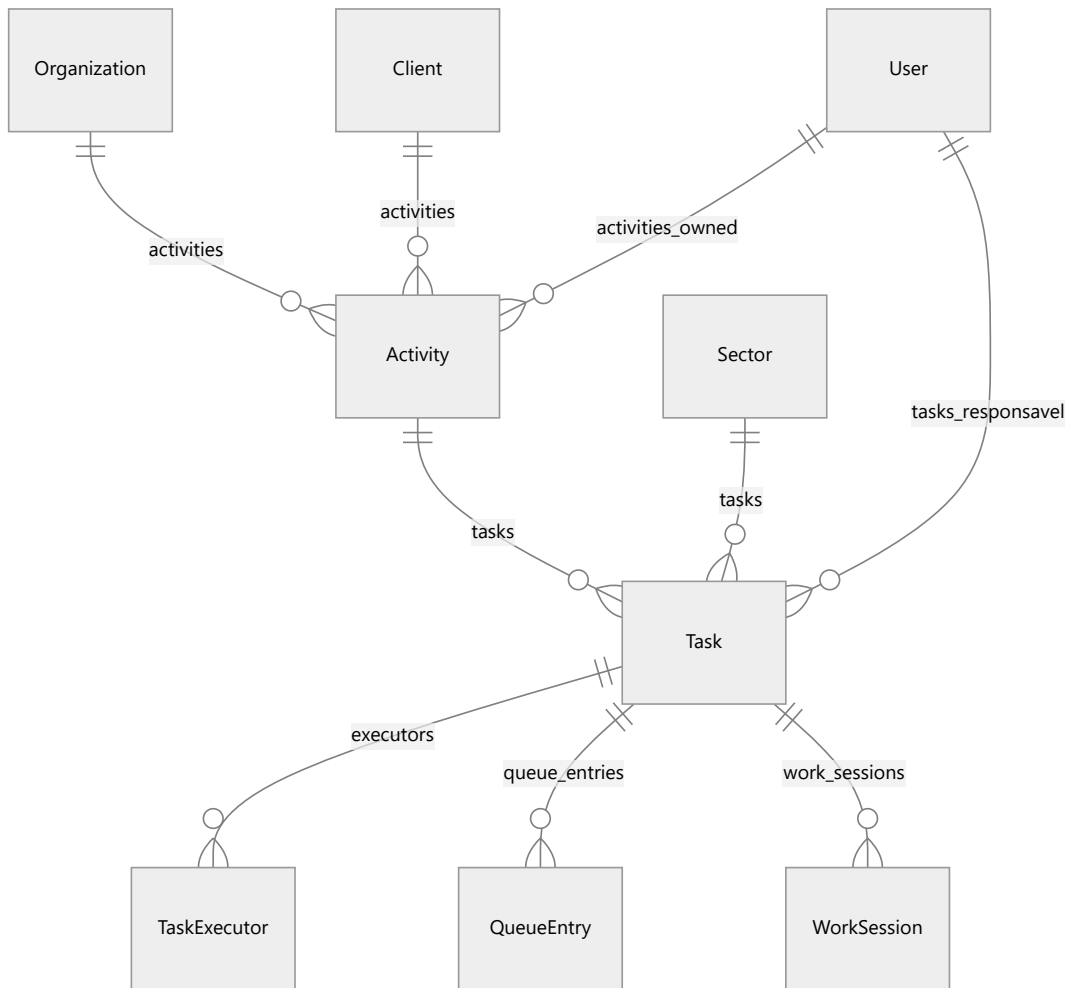
1. Visão do núcleo

Pessoas, organização e acesso



(`AccessProfile` é `acessos.Profile`; `Profile` é `accounts.Profile`.)

Trabalho: atividade, tarefa e fila



2. core

Todos, exceto `Organization`, têm `organization → Organization` (CASCADE).

Model	Campos relevantes	Constraints / regras
<code>Organization</code>	<code>name</code> (único), <code>is_active</code>	O tenant.
<code>Company</code>	<code>name</code> , <code>document</code> , <code>is_active</code>	Único por (org, name). Empresa operacional interna (Regras 05 §9).
<code>Sector</code>	<code>name</code> , <code>description</code> , <code>is_active</code> , <code>created_by</code>	Único por (org, name). Nunca fixo no código.
<code>Client</code>	<code>name</code> , <code>document</code> , <code>phone</code> , <code>email</code> , <code>address</code> , <code>is_active</code>	Único por (org, name). Quem solicita o serviço; diferente de <code>Company</code> .
<code>Site</code> (obra)	<code>client → Client</code> (null, <code>sites</code>), <code>name</code> , <code>is_active</code>	Regras 10 §11. Não tem mais <code>company</code> (migration 0010).
<code>CostCenter</code>	<code>site → Site</code> (null, <code>cost_centers</code>), <code>name</code> , <code>is_active</code>	
<code>Tag</code>	<code>name</code> , <code>color</code> (hex da paleta de 36 cores), <code>is_active</code>	Único por (org, name).
<code>TaskStage</code>	<code>name</code> , <code>order</code> , <code>color</code> , <code>is_active</code>	Único por (org, name). Coluna do Kanban de tarefas; nunca altera <code>Task.status</code> .
<code>ActivityStage</code>	<code>name</code> , <code>order</code> , <code>color</code> , <code>is_active</code>	Único por (org, name). Coluna do Kanban de atividades; nunca altera <code>Activity.status</code> .

Model	Campos relevantes	Constraints / regras
WorkflowStatus	domain (activity/task), name, description, behavior (código nativo que imita), color, is_active	Único por (org, domain, name). Status definidos pela organização, preparados para telas futuras.
EnumColor	domain (activity_status, task_status, activity_urgency), code, color, label, description, is_hidden, updated_by	Único por (org, domain, code). Sobrescreve cor/rótulo de um valor de enum fixo; o significado do código não muda.

A paleta e as cores padrão ficam em `core/colors.py` (PALETTE, DEFAULTS), espelhadas em `static/js/color-utils.js`.

3. accounts

Model	Campos relevantes	Constraints / regras
Profile	user 1:1 (profile), organization (null, PROTECT, members), phone, main_sector (null, SET_NULL)	Criado vazio por signal ao criar um User. main_sector é só o filtro padrão das telas (Regras 05 §29).
UserSector	user (sector_memberships), sector (user_memberships), role (MEMBRO, GESTOR), joined_at, removed_at	Único por (user, sector) enquanto removed_at é nulo — sair e voltar abre novo período. Ser GESTOR não concede autorização por si só.
EmailVerification	user 1:1, code (6 dígitos), expires_at, attempts, confirmed_at	CODE_TTL =15 min, RESEND_COOLDOWN =60 s, MAX_ATTEMPTS =5. issue(user) substitui o código anterior.

4. acessos

Model	Campos relevantes	Constraints / regras
ActionGroup	key, name, order, is_active	Só organiza a interface.
Action	group, key (único, ex. fila.reordenar), name, description, is_sensitive, is_active	Definida pelo produto em <code>catalog.py</code> , nunca pelo cliente. is_sensitive só destaca na UI/auditoria.
Profile (perfil de acesso)	organization (profiles), name, actions (M2M via ProfileAction), is_active	Único por (org, name).
ProfileAction	profile, action	Único por (profile, action).
Scope	organization, type, company / sector / site / cost_center (opcionais), relation, is_active	clean() exige exatamente o campo do tipo; chamado no save(). Linhas reutilizáveis.
UserProfile	user (access_profiles), profile (assignments), scope (PROTECT), is_active	Único por (user, profile, scope) enquanto ativo.
UserAction (concessão direta)	user (direct_actions), action (direct_grants), scope (PROTECT), is_active	Único por (user, action, scope) enquanto ativo. Só positiva — não existe "negar".

Scope.type: ORGANIZACAO, EMPRESA, SETOR, OBRA, CENTRO_CUSTO, RELACIONAL. Scope.relation: MINHAS_ATIVIDADES, MINHAS_TAREFAS, MEUS_SETORES, SETORES_GERENCIADOS. O significado de cada um está em [05_AUTORIZACAO.md](#).

5. activities

Atividade

Model	Campos relevantes
Activity	code (único, ATV-AAAA-NNNNN, gerado no primeiro save — Regra 12), title (o resultado esperado), description (HTML sanitizado), internal_notes, status, urgency, organization, client, company, site, cost_center, sector (grupo designado), stage → ActivityStage, owner (PROTECT; nulo só em rascunho), requested_by (solicitante interno), created_by, process_version (PROTECT), tags (M2M), requested_deadline, first_action_at, completed_at, completion_outcome, cancelled_at, cancelled_reason, reopened_at

Model	Campos relevantes
OwnerChangeLog	activity (owner_changes), previous_owner, new_owner, changed_by — nunca sobrescrito
ActivityPendency	activity (pendencies), reason, status, comment, decision_deadline, notify_client, previous_owner, approver, opened_by, resolved_by, resolution_comment
ActivityMessage	activity (messages), author, body, kind
ActivityAttachment	activity (attachments), file (storage em ACTIVITY_FILES_ROOT), original_name, uploaded_by
ReturnReason	organization, name, is_active — único por (org, name)

Enums:

Enum	Valores
Activity.Status	RASCUNHO, ABERTA (padrão), EM_ANDAMENTO, BLOQUEADA, PENDENTE, CONCLUIDA, CANCELADA
Activity.Urgency	BAIXA, MEDIA (padrão), ALTA
Activity.CompletionOutcome	SUCESSO, CONCLUIDO_COM_PENDENCIAS, DECLINADO, CANCELADO
ActivityPendency.Reason	APROVACAO_GESTOR, AJUSTES_REVISOES (as duas exigem aprovação), MATERIAL, INFORMACOES_CLIENTE
ActivityPendency.Status	ABERTA, APROVADA, ENCERRADA
MessageKind (atividade e tarefa)	NORMAL, DECISAO, RISCO, COMPROMISSO, IMPEDIMENTO

Tarefa

Model	Campos relevantes
Task	activity (CASCADE, tasks), sector (PROTECT), stage → TaskStage, responsavel (PROTECT, obrigatório desde a migration 0015), depends_on → Task, order, title, description, status, requested_deadline, committed_deadline, first_action_at, completed_at, cancelled_at, overdue_notified_at, tags
TaskExecutor (participante)	task (executors), user (tasks_executed), added_by, removed_at (remoção lógica)
TaskAssignment	task (assignments), user, assigned_by, status (PENDENTE, ACEITA, RECUSADA), reason → ReturnReason, observation
TaskResponsavelChangeLog	task (responsavel_changes), previous_responsavel, new_responsavel, changed_by
WorkSession	task, user, started_at, ended_at, is_manual — propriedade duration
TaskBlock	task (blocks), reason, observation, started_by/at, ended_by/at
SectorTransfer	task, from_sector, to_sector, moved_by, note
TaskReturn	task, from_sector, to_sector, reason → ReturnReason, observation, returned_by
TaskMessage	task (messages), author, body, kind
TaskChecklistItem	task (checklist_items), text, is_done, order, done_by, done_at

Task.Status: NAO_INICIADA (padrão do model, mas nenhum fluxo deixa a tarefa nele), DISPONIVEL, EM_FILA, EM_EXECUCAO, BLOQUEADA, DEVOLVIDA, CONCLUIDA, CANCELADA.

Fila e prazo

Model	Campos relevantes
QueueEntry	task, sector, position, queue_size_at_entry, entered_at, left_at — uma linha por passagem pela fila (Regras 04 §37)
QueuePositionChange	queue_entry, old/new_position, old/new_total, reason (AUTOMATICA_CONCLUSAO, AUTOMATICA_ENTRADA, MANUAL, ENTRADA_PRIORITARIA), changed_by (nulo = automático)

Model	Campos relevantes
DeadlineProposal	task, proposed_deadline, proposed_by, status (PENDENTE, ACEITO, RECUSADO), decided_by, decision_note
DeadlineConflict	task, proposal 1:1, status (ABERTO, RESOLVIDO), resolution_note, resolved_by

Activity, Task e DeadlineConflict ainda declaram `Meta.permissions` do Django (`can_change_owner`, `can_reorder_queue` etc.). São legado: a autorização real é o motor do `acessos`.

6. processes

Model	Campos relevantes
ActivityType	organization, name — único por (org, name)
Process	organization, company (PROTECT), activity_type, name, description, is_active — único por (company, name); propriedades <code>published_version</code> , <code>draft_version</code>
ProcessVersion	process (versions), number, status (RASCUNHO, PUBLICADO, SUBSTITUIDO), output_description, output_evidence_type (ARQUIVO, LINK, CHECKLIST, CONFIRMACAO), published_by/at — só editável em rascunho
ProcessInput	version (inputs), name, input_type (TEXTO, ARQUIVO, DATA, NUMERO, LINK, SELECAO), is_required, source (SOLICITANTE, EXECUTOR, TERCEIRO), order
ProcessCriterion	version (criteria), name, is_required, order
ProcessStep	version (steps), sector, name, order, depends_on_previous
ActivityInputValue	activity, process_input, value, is_received — único por (activity, input)
ActivityCriterionCheck	activity, process_criterion, is_met — único por (activity, criterion)

7. notifications e audit

Model	Campos relevantes
Notification	recipient (notifications), event_type, actor, activity, task, title, message, url, is_read, read_at — índice (recipient, is_read)
AuditLog	user, activity, task, target_user (eventos de segurança), action, field_name, old_value, new_value, reason, timestamp — índices (activity, timestamp) e (task, timestamp)

Os valores de `Notification.EventType` e `AuditLog.Action` estão em [08_NOTIFICACOES_E_AUDITORIA.md](#).

8. Migrations

- Cada app tem suas migrations em `<app>/migrations/`.
- `activities` tem dois `0005_*` unidos por `0006_merge_*`.
- Migrations de dados que valem conhecer:
- `activities/0012_fix_stuck_em_execucao_tasks`: corrige tarefas `EM_EXECUCAO` sem sessão aberta.
- `activities/0014_populate_task_responsavel` e `0015_task_responsavel_not_null`: preenchem e tornam obrigatório o responsável.
- `core/0006–0007`: migram cores de tag para hex.
- `core/0010_remove_site_company_site_client`: obra passa a ter cliente em vez de empresa.
- Depois de mudar um model: `python manage.py makemigrations` e confira com `python manage.py makemigrations --check --dry-run` antes de commitar.

05 — Autorização

Como a LPS decide se uma pessoa pode fazer algo. O modelo é **SUJEITO + AÇÃO + ESCOPO + ORIGEM**: toda permissão é uma ação, vale em um escopo e tem uma origem explicável (um perfil ou uma concessão direta). Nega por padrão e não existe "negar explícito". Regras de produto em `Regras/05_USUARIOS_SETORES_E_AUTORIZACOES.md`; implementação em `acessos/`.

1. Peças

Peça	Onde	O que é
Catálogo de ações	<code>acessos/catalog.py</code>	Constantes (<code>catalog.TAREFA_INICIAR = "tarefa.iniciar"</code>) e a lista <code>GROUPS</code> . Única fonte da verdade : o código referencia as constantes; <code>seed_acoes</code> grava no banco.
Action / ActionGroup	<code>acessos/models.py</code>	O catálogo materializado. Uma ação só existe se a aplicação tiver comportamento correspondente (doc 05 §14).
Perfil de acesso (<code>acessos.Profile</code>)	por organização	Conjunto reutilizável de ações (<code>ProfileAction</code>).
Scope	por organização	"Onde vale": a organização toda, uma empresa, um setor, uma obra, um centro de custo ou uma relação.
<code>UserProfile</code>	por pessoa	Atribuição: pessoa + perfil + escopo.
<code>UserAction</code>	por pessoa	Concessão direta: pessoa + ação + escopo. Exceção pontual, não a forma padrão (doc 05 §27).

Participar de um setor (`accounts.UserSector`), mesmo como GESTOR, **não concede nada** por si só. Só alimenta os escopos relacionais `MEUS_SETORES` e `SETORES_GERENCIADOS` e os destinatários de notificações.

2. Escopos

	Cobre o recurso quando...
<code>ORGANIZACAO</code>	sempre (dentro da organização)
<code>EMPRESA</code>	<code>recurso.company_id == scope.company_id</code>
<code>SETOR</code>	<code>recurso.sector_id == scope.sector_id</code> (para tarefa, é o setor da tarefa)
<code>OBRA</code>	<code>recurso.site_id == scope.site_id</code>
<code>CENTRO_CUSTO</code>	<code>recurso.cost_center_id == scope.cost_center_id</code>
<code>RELACIONAL + MINHAS_ATIVIDADES</code>	a pessoa é dona da atividade
<code>RELACIONAL + MINHAS_TAREFAS</code>	a pessoa é participante ativa (<code>TaskExecutor</code>) da tarefa
<code>RELACIONAL + MEUS_SETORES</code>	o setor do recurso é um dos setores da pessoa
<code>RELACIONAL + SETORES_GERENCIADOS</code>	o setor do recurso é um que a pessoa gerencia (GESTOR)

`MINHAS_TAREFAS` olha só os **participantes**; o `responsavel` da tarefa não entra nesse conjunto.

O "endereço" de um recurso é montado por `ResourceContext.of(recurso)` em `acessos/services.py`:

Recurso	Endereço
<code>Activity</code>	org, empresa, setor designado, obra, centro de custo, dono
<code>Task</code>	setor da tarefa + resto da atividade + participantes ativos

Recurso	Endereço
<code>QueueEntry</code>	setor da fila + dados da atividade + participantes
<code>DeadlineProposal</code> , <code>DeadlineConflict</code>	o da tarefa
<code>Sector</code> , <code>Company</code> , <code>Site</code> , <code>CostCenter</code>	org + o próprio id
outro model com <code>organization_id</code>	só a org
qualquer outra coisa	<code>unresolved</code> → nega
<code>None</code>	vazio: só escopos <code>ORGANIZACAO</code> (e relacionais) casam

Para **criar** algo que ainda não existe, os serviços passam `ResourceContext.for_new(organization=..., sector=..., ...)`.

3. Como `AuthorizationService.can()` decide

```
AuthorizationService.can(user, catalog.TAREFA_INICIAR, task)
```

1. Superusuário ativo → **permitido** (`is_platform_admin`). É o operador da plataforma; ainda precisa de organização no `Profile` para usar as telas.
2. Usuário anônimo, inativo ou sem organização → **negado**.
3. Monta o endereço com `ResourceContext.of(resource)`.
4. **Teto de tenant**: recurso de outra organização → **negado** (doc 05 §43).
5. Busca atribuições ativas (`UserProfile`) cujo perfil ativo contém a ação ativa, e concessões diretas ativas (`UserAction`) para a ação.
6. Mantém as que têm escopo ativo que **cobre** o endereço (`scope_contains`).
7. Sobrou alguma → **permitido**.

Outras funções úteis de `AuthorizationService`:

Função	Uso
<code>require(user, key, resource)</code>	Levanta <code>AuthorizationError</code> com mensagem legível.
<code>grants_for(user, key, resource)</code>	Lista de <code>Grant</code> com a origem de cada permissão.
<code>explain(user, key, resource)</code>	Motivos em texto ("Perfil: Gestor de Setor · Escopo: Setor Compras").
<code>effective_actions(user)</code>	Todas as ações da pessoa com origem — usado nas telas de segurança.
<code>can_anywhere(user, key)</code>	Permitido em algum lugar (org ou algum setor)? Usado para menus e listas.
<code>accessible_sector_ids(user, key)</code>	Setores em que a ação vale.

Os escopos de setor são memoizados na instância do usuário; qualquer mudança em `UserSector` invalida o cache via signal (`invalidate_sector_cache`).

4. Onde a verificação acontece

Camada	Mecanismo
Views de <code>core</code> (cadastros, usuários, segurança)	<code>ActionRequiredMixin</code> com <code>required_action = catalog.X</code> . Deve vir depois de <code>OrganizationRequiredMixin</code> nas bases. Nega com <code>PermissionDenied</code> . Escopo padrão: organização (<code>get_scope_object()</code> retorna <code>None</code>).
Serviços de <code>activities</code>	<code>require_action(user, catalog.X, recurso)</code> no início de cada método; converte a negação em <code>ActivityError</code> .
Serviços de <code>processes</code>	<code>_require(...)</code> com endereço org + empresa; levanta <code>ProcessError</code> .
Menu lateral	<code>acessos.context_processors.navigation</code> — só UX , não protege nada.

Algumas ações do catálogo ainda não são verificadas em nenhum ponto (ex.: `atividade.visualizar`) — ver [13_PENDENCIAS_CONHECIDAS.md](#).

5. Administração pela interface

Tela	URL	Ação exigida
Matriz de permissões (perfis × ações)	/permissoes/	seguranca.gerir_perfis
Salvar ações de um perfil	/permissoes/<pk>/salvar/	seguranca.gerir_autorizacoes
Criar/editar perfil	/perfis/novo/, /perfis/<pk>/	seguranca.gerir_perfis
Acessos de uma pessoa (perfis e concessões, com origem)	/usuarios/<pk>/acessos/	seguranca.gerir_autorizacoes
Concessão direta	/usuarios/<pk>/concessoes/	seguranca.gerir_autorizacoes

Todas as alterações passam por `AccessService` e são auditadas (`PROFILE_CREATED`, `PROFILE_ASSIGNED`, `ACTION_GRANTED` etc., com `target_user`). `AccessService.set_profile_actions` só altera as ações que estavam **visíveis** na tela, para um filtro na matriz não apagar o resto.

6. Perfis sugeridos

Criados por `seed_lps_demo` a partir de `SUGGESTED_PROFILES`. São ponto de partida — a organização pode renomear e alterar (doc 05 §11).

Perfil	Resumo
Colaborador	Ver e criar atividades; assumir e marcar pendente; executar tarefas (assumir, aceitar, recusar, iniciar, pausar, retomar, concluir, devolver, bloquear); ver a própria posição na fila; propor/aceitar/recusar prazo; conversar; ver e aplicar processos; gerir clientes.
Gestor de Setor	Tudo do Colaborador + ver todas as atividades, editar, aprovar pendência; criar/editar/atribuir tarefas, mover de setor, lançar tempo manual; fila completa e reordenar; resolver conflito de prazo; métricas e auditoria; todas as ações de processo; estágios, tags e cores.
Administrador	Todas as ações do catálogo.

7. Receita: adicionar uma ação nova

- Em `acessos/catalog.py`, crie a constante (`MINHA_ACAO = "grupo.minha_acao"`) e adicione a tupla (`chave`, `nome`, `descrição`, `sensível`) ao grupo certo em `GROUPS`. Se fizer sentido, inclua em `SUGGESTED_PROFILES`.
- Use a constante no código: - view de `core: required_action = catalog.MINHA_ACAO`; - serviço: `require_action(user, catalog.MINHA_ACAO, recurso)`.
- Se o recurso for um model novo, ensine `ResourceContext.of` a endereçá-lo — senão ele só terá o escopo da organização (ou será negado, se não tiver `organization_id`).
- Rode `python manage.py seed_acoes` (local e no deploy) para gravar a ação.
- Teste com os helpers de `acessos/testing.py` (`grant_action(user, catalog.MINHA_ACAO, organization=org)`).

Para **remover** uma ação: tire do catálogo e rode `seed_acoes`; ela fica `is_active=False` e deixa de conceder, mas o histórico é preservado.

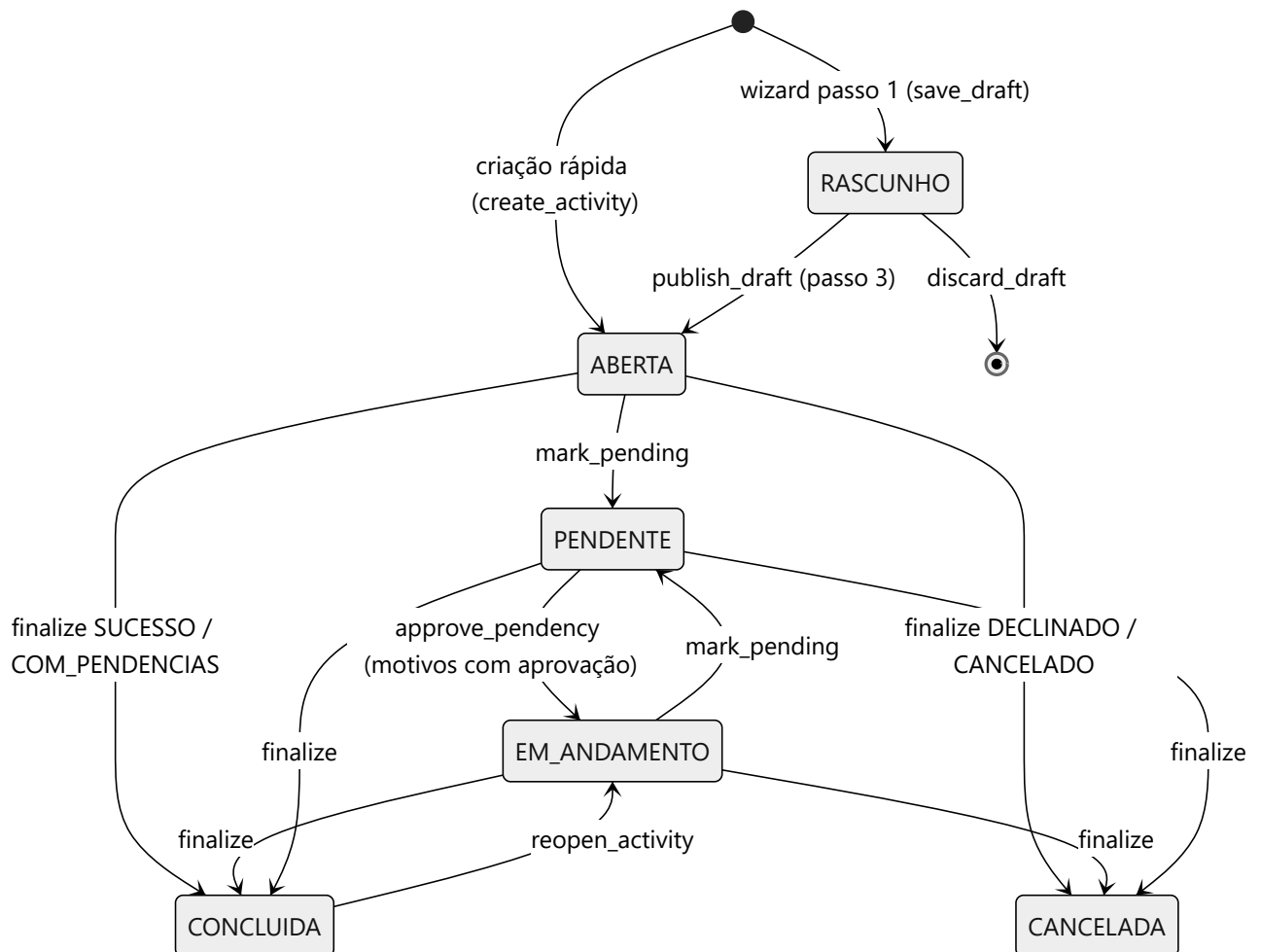
06 — Atividades e tarefas

O núcleo operacional da LPS, em `activities/`. Uma **atividade** é um resultado a alcançar e tem exatamente um dono. Uma **tarefa** é uma unidade de trabalho de uma atividade, endereçada a um setor, com um responsável e participantes. As regras de produto estão em `Regras/02_ATIVIDADES_TAREFAS_E_FLUXOS.md` e `Regras/03_FILAS_PRAZOS_E_ESCALONAMENTO.md`.

Toda a regra está em `activities/services.py` (`ActivityService`, `TaskService`, `QueueService`, `DeadlineService`, `MessageService`, `ActivityAttachmentService`). Cada método verifica a ação do catálogo, valida a transição, grava, audita e notifica.

1. Atividade

1.1 Estados



- Nenhum fluxo leva `ABERTA` → `EM_ANDAMENTO` automaticamente (iniciar uma tarefa só marca `first_action_at`). `EM_ANDAMENTO` hoje só é alcançado por reabertura ou aprovação de pendência.
- `BLOQUEADA` existe no enum, mas nenhum serviço coloca uma atividade nesse estado.
- Pendências sem aprovação (`MATERIAL`, `INFORMACOES_CLIENTE`) só saem de `PENDENTE` pela finalização.

Esses três pontos estão em [13_PENDENCIAS_CONHECIDAS.md](#).

1.2 Criação

Wizard em 3 passos (/atividades/nova/):

Passo	URL	Campos	Serviço
1. Essencial	atividades/nova/ (?pk= retoma um rascunho)	título, cliente, obra, setor, dono, prazo solicitado, urgência	ActivityService.save_draft — exige atividade.criar; cria em RASCUNHO (sem auditoria nem notificação). O código ATV-AAAA-NNNN já é gerado aqui.
2. Contexto	atividades/<pk>/nova/contexto/	empresa, centro de custo, solicitante, endereço, tags	salva o form diretamente
3. Detalhes	atividades/<pk>/nova/detalhes/	descrição, notas internas, anexos, resumo	publish_draft — re-verifica atividade.criar, exige título real e dono, vai para ABERTA, audita CREATE, notifica dono e criador (ACTIVITY_CREATED) e processa @menções. "Publicar e criar tarefas" leva direto para a criação de tarefas.

discard_draft (só o criador) apaga o rascunho e seus anexos.

Criação rápida (atividades/nova-rapida/, JSON): só título; dono = quem criou; nasce ABERTA. Usada pelos seletores que criam atividade sem sair da tela.

1.3 Dono, edição e pendência

Operação	Serviço	Ação	Efeitos
Trocar dono	change_owner	atividade.alterar_dono	OwnerChangeLog, audita OWNER_CHANGED, notifica antigo e novo dono
Assumir do grupo	claim	atividade.assumir	atividade precisa ter setor designado
Editar	update_activity	atividade.editar	audita UPDATE por campo; bloqueado se concluída/cancelada; nova descrição reprocessa menções
Marcar pendente	mark_pending	atividade.marcar_pendente	de ABERTA / EM_ANDAMENTO; motivo e comentário obrigatórios; audita PENDING_OPENED e gera mensagem
Aprovar pendência	approve_pendency	atividade.aprovar_pendencia	pendência APROVADA, atividade EM_ANDAMENTO, dono volta ao anterior, audita PENDING_APPROVED, notifica ACTIVITY_APPROVED

Pendência **com aprovação** (APROVACAO_GESTOR, AJUSTES_REVIS0ES): exige prazo de decisão e setor designado com gestor. O primeiro gestor do setor vira dono temporário (com OwnerChangeLog); todos os gestores recebem ACTIVITY_APPROVAL_NEEDED por notificação e e-mail; o dono anterior recebe ACTIVITY_PENDING.

Pendência **sem aprovação**: o dono recebe ACTIVITY_PENDING. Em INFORMACOES_CLIENTE com "avisar cliente", o cliente recebe e-mail (Client.email).

1.4 Finalização e reabertura

finalize(outcome, comment) é o popup único de encerramento; comentário obrigatório.

Resultado	Ação	Estado final	Pré-condição
SUCESSO, CONCLUIDO_COM_PENDENCIAS	atividade.concluir	CONCLUIDA	nenhuma tarefa aberta
DECLINADO, CANCELADO	atividade.cancelar	CANCELADA (comentário vira cancelled_reason)	—

Pendências abertas viram ENCERRADA; audita COMPLETE / CANCEL; notifica o dono. reopen_activity (atividade.reabrir) leva CONCLUIDA → EM_ANDAMENTO com motivo obrigatório (REOPEN). Os endpoints antigos complete_activity / cancel_activity ainda existem.

1.5 Anexos, mensagens e menções

- Anexos** (ActivityAttachmentService): adicionar exige atividade.editar (ou ser o criador do próprio rascunho); remover exige atividade.editar e apaga o arquivo. Auditado como UPDATE no campo "anexo". Local de gravação em 03_CONFIGURACAO.md.

2.3 Responsável x participantes

- **Responsável:** um único usuário (`Task.responsavel`), responde pela conclusão.
- **Participantes:** `TaskExecutor` , podem ser vários; nunca incluem o responsável.

Operação	Serviço	Ação	Efeito
Assumir (entrar como participante)	<code>add_executor</code> em si mesmo	<code>arefa.assumir</code>	participante imediato, <code>EXECUTOR_ADDED</code>
Atribuir a outra pessoa	<code>add_executor</code>	<code>arefa.atribuir</code>	cria <code>TaskAssignment</code> <code>PENDENTE</code> , notifica <code>TASK_ASSIGNMENT_PENDING</code>
Aceitar atribuição	<code>accept_assignment</code>	<code>arefa.aceitar</code>	vira participante (<code>ASSIGNMENT_ACCEPTED</code> + <code>EXECUTOR_ADDED</code>)
Recusar atribuição	<code>reject_assignment</code>	<code>arefa.recusar</code>	motivo (<code>ReturnReason</code>) obrigatório; notifica quem atribuiu
Remover participante	<code>remove_executor</code>	<code>arefa.assumir</code> (a si) / <code>arefa.atribuir</code> (outro)	remoção lógica (<code>removed_at</code>)
Trocar responsável	<code>change_responsavel</code>	<code>arefa.alterar_responsavel</code>	<code>TaskResponsavelChangeLog</code> , <code>RESPONSABLE_CHANGED</code> , notifica

2.4 Execução e tempo

- `start` (`arefa.iniciar`): só responsável ou participante; abre uma `WorkSession` da pessoa (se ainda não houver) e vai para `EM_EXECUCAO` . Marca `first_action_at` na tarefa e na atividade. Uma pessoa **pode** ter sessões abertas em várias tarefas ao mesmo tempo (Regras 04 §115, revisada).
- `pause` (`arefa.pausar`): fecha a sessão da pessoa; se não restar nenhuma sessão aberta, volta a `EM_FILA` .
- `resume` (`arefa.retomar`): delega para `start` .
- `log_manual_time` (`tempo.lancar_manual`): cria `WorkSession` com `is_manual=True` ; sem datas futuras ou invertidas.
- O cronômetro da barra superior lista as sessões abertas do usuário (context processor `my_active_sessions`).

2.5 Bloqueio, mudança de setor e devolução

Operação	Serviço	Ação	Efeito
Bloquear	<code>block</code>	<code>arefa.bloquear</code>	motivo obrigatório; <code>TaskBlock</code> ; <code>BLOQUEADA</code> ; notifica dono (<code>TASK_BLOCKED</code>)
Desbloquear	<code>unblock</code>	<code>arefa.bloquear</code>	fecha o bloqueio; <code>EM_FILA</code> ; <code>TASK_UNBLOCKED</code>
Enviar a outro setor	<code>move_to_sector</code>	<code>arefa.mover_setor</code>	sai da fila antiga (renumera), <code>SectorTransfer</code> , entra na fila nova; notifica o novo setor
Devolver	<code>return_task</code>	<code>arefa.devolver</code>	motivo (<code>ReturnReason</code>) obrigatório; <code>TaskReturn</code> ; depois <code>move_to_sector</code> ; notifica os dois setores e o dono

2.6 Fila

- Cada setor tem uma fila; a posição é sempre mostrada como "N de M".
- `QueueService.enqueue` põe no fim; `renumber` fecha buracos na conclusão, cancelamento e saída de setor (motivo `AUTOMATICA_CONCLUSAO`).
- `QueueService.reorder` (`fila.reordenar` no setor): a tarefa movida recebe `MANUAL` , as deslocadas `AUTOMATICA_ENTRADA` ; cada mudança é auditada (`QUEUE_POSITION_CHANGED`) e o dono da atividade é notificado.
- `/fila/` mostra a fila completa só com `fila.visualizar_completa` no setor; sem ela, só as entradas em que a pessoa é dona, responsável ou participante.

2.7 Prazo

- **Solicitado** (`requested_deadline`): quem pede. **Comprometido** (`committed_deadline`): o que o executor assumiu.
- `DeadlineService.propose` (`prazo.propor`) cria `DeadlineProposal` `PENDENTE` e notifica o dono.
- Só o **dono da atividade** aceita (`prazo.aceitar` , grava o prazo comprometido) ou recusa (`prazo.recusar`).

- Recusar abre um `DeadlineConflict` e notifica setor e dono (`DEADLINE_CONFLICT`). `resolve_conflict` exige `escalonamento.resolver`. O motor de escalonamento automático é D1 (Regras 10 §57) — hoje só registra e notifica.

2.8 Checklist

`TaskCheckListItem`, editado via Ajax (`static/js/checklist.js`). Adicionar e remover exigem `tarefa.editar`; marcar/desmarcar também é permitido ao responsável ou participante. Guarda `done_by` / `done_at`. Não é auditado.

2.9 Atraso

`python manage.py check_overdue_tasks` → `TaskService.mark_overdue_tasks()`:

1. seleciona tarefas com `committed_deadline` no passado, `overdue_notified_at` nulo e status `DISPONIVEL`, `EM_FILA`, `EM_EXECUCAO` ou `BLOQUEADA`;
2. notifica membros e gestores do setor, dono da atividade e responsável (`TASK_OVERDUE`) e envia e-mail (`notifications/email/task_overdue.txt`);
3. grava `overdue_notified_at` — cada tarefa é avisada uma única vez.

Precisa ser agendado externamente (não há Celery/worker).

2.10 Kanban de tarefas

Mesma ideia das atividades: colunas = `TaskStage` da organização, mover altera só `stage`. A ordem das colunas é arrastável em Cadastros → Estágios (`TaskStageReorderView`).

3. Processos (`processes/`)

Modelos reutilizáveis e versionados de "como fazer" um tipo de atividade, por empresa (Regras 11 / `Regras/12_PROCESSOS_INPUTS_OUTPUTS_E_CRITERIOS_DE_ACEITE.md`).

- `ProcessService.create` cria o processo e a **v1 em rascunho** (`processo.criar`).
- Rascunho é editável (`processo.editar_rascunho`): inputs, output com tipo de evidência, critérios de aceite e fluxo linear de etapas por setor.
- `publish` (`processo.publicar`) exige output e ao menos uma etapa; a versão publicada anterior vira `SUBSTITUIDO`. Versão publicada não muda mais.
- `create_new_version` (`processo.criar_versao`) copia a publicada para v(n+1) em rascunho; recusa se já houver rascunho.
- `set_active` (`processo.inativar`).

Ainda não ligado às atividades. `Activity.process_version`, `ActivityInputValue` e `ActivityCriterionCheck` existem no banco, mas nenhum código aplica um processo a uma atividade nem gera tarefas a partir das etapas. `processo.aplicar` só existe no catálogo.

07 — Contas e autenticação

Como uma pessoa passa a existir na LPS e como entra. Especificação das telas em `Telas/09_01_LOGIN.md`, `09_02_CRIAR_CONTA.md`, `09_03_CONFIRMAR_EMAIL.md` e `09_04_CADASTRO_NOVO_USUARIO_LPS.md`. Código em `accounts/` e nas telas de usuário de `core/`.

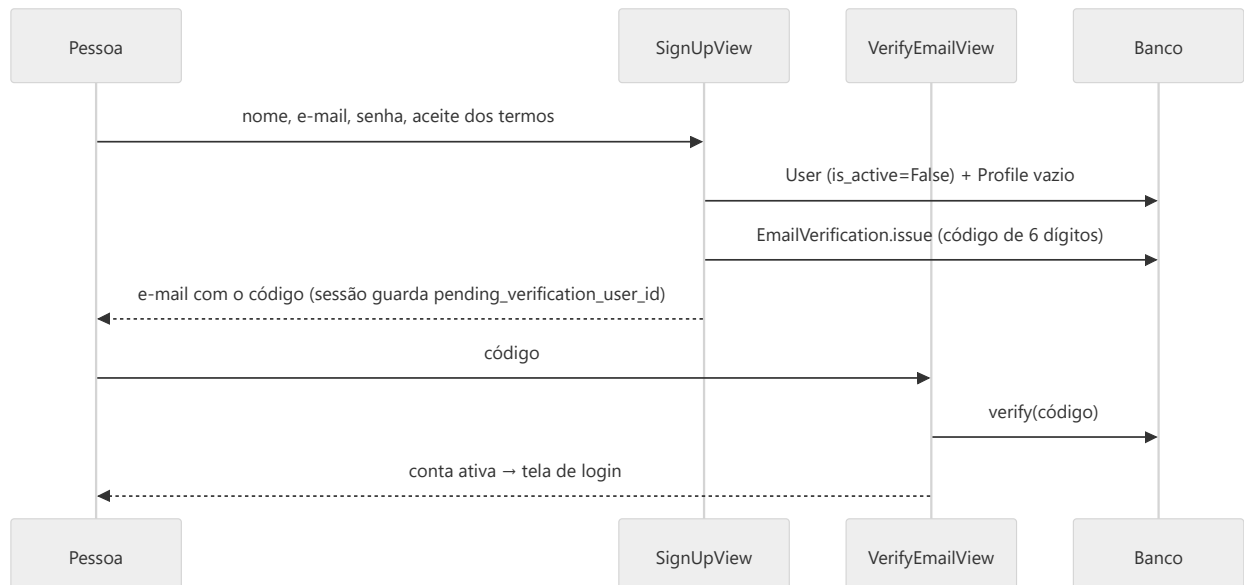
1. Duas formas de criar uma pessoa

	Autocadastro (<code>/accounts/criar-conta/</code>)	Cadastro pelo administrador (<code>/usuarios/novo/</code>)
Quem faz	a própria pessoa	quem tem <code>usuario.editar</code>
Conta nasce	inativa até confirmar o e-mail	ativa, com senha definida pelo admin
<code>username</code>	gerado (<code>conta-<20 hex></code>)	digitado pelo admin
Organização	nenhuma	a do admin (campo obrigatório, travado nela)
Setores	nenhum	escolhidos no formulário (<code>AccessService.sync_user_sectors</code>)
Auditoria	não	<code>USER_CREATED</code> (+ <code>SECTORS_CHANGED</code>)

Em ambos os casos o signal `accounts.signals` cria o `Profile` vazio no `post_save` do `User`; o `UserFormView` preenche a organização em seguida.

Uma conta criada pelo autocadastro consegue logar depois de confirmar o e-mail, mas **toda tela da LPS a redireciona para `/accounts/me/`** até um administrador vinculá-la a uma organização (hoje, pelo Admin do Django — a tela `/usuarios/` só lista pessoas que já são da organização).

2. Autocadastro e confirmação de e-mail



- `SignUpForm` (`accounts/forms.py`): e-mail único sem diferenciar maiúsculas; valida a senha com os validadores do Django; aceite dos termos obrigatório.
- `EmailVerification` (`accounts/models.py`):
- código de 6 dígitos, válido por **15 minutos** (`CODE_TTL`);
- no máximo **5 tentativas** (`MAX_ATTEMPTS`);
- reenviar só depois de **60 segundos** (`RESEND_COOLDOWN`); o novo código substitui o anterior.

- A tela de confirmação mostra o e-mail mascarado e usa a sessão (`PendingVerificationMixin`) para saber qual conta está pendente; sem ela, volta para o cadastro.
- `confirmar-email/alterar/` troca o e-mail da conta pendente (também único) e emite um novo código.
- Em dev, o e-mail com o código aparece no console do `runserver`.

URL	Nome	View
<code>/accounts/criar-conta/</code>	<code>signup</code>	<code>SignUpView</code>
<code>/accounts/confirmar-email/</code>	<code>verify-email</code>	<code>VerifyEmailView</code>
<code>/accounts/confirmar-email/reenviar/</code> (POST)	<code>verify-email-resend</code>	<code>ResendVerificationCodeView</code>
<code>/accounts/confirmar-email/alterar/</code>	<code>verify-email-change</code>	<code>ChangeVerificationEmailView</code>

3. Login

- `/accounts/login/` usa `LoginView` com `EmailAuthenticationForm` (o campo "username" vira um campo de e-mail).
- `accounts.auth_backends.EmailBackend` é o **único** backend: busca `User` por `email__iexact`, confere a senha e `user_can_authenticate` (contas inativas são recusadas). Se houver dois usuários com o mesmo e-mail, o login é recusado.
- O `username` continua existindo como identificador interno e para @menções.
- Após o login: `home (/)`. Logout: `/accounts/logout/` → login.

4. Recuperação de senha

Views nativas do Django, com templates próprios em `templates/accounts/`:

URL	Nome
<code>/accounts/esqueci-senha/</code>	<code>password_reset</code>
<code>/accounts/esqueci-senha/enviado/</code>	<code>password_reset_done</code>
<code>/accounts/redefinir-senha/<uidb64>/<token>/</code>	<code>password_reset_confirm</code>
<code>/accounts/redefinir-senha/concluido/</code>	<code>password_reset_complete</code>

Corpo e assunto do e-mail: `templates/registration/password_reset_email.html` e `password_reset_subject.txt`. A troca pelo próprio usuário **não** é auditada; `AuditLog.PASSWORD_RESET` só é gravado quando um admin define nova senha em `/usuarios/<id>/`.

5. Perfil e superusuário

- `/accounts/me/` (`profile`): a pessoa edita telefone e setor principal (`ProfileForm`). A organização aparece só para leitura.
- Superusuário: criado por `createsuperuser` (local) ou `ensure_superuser` (deploy). Passa por todas as verificações de autorização, mas precisa de organização no `Profile` para usar as telas da LPS — e de **e-mail** para conseguir logar.

08 — Notificações e auditoria

O que cada evento gera: registro de auditoria (`audit.AuditLog`), notificação in-app (`notifications.Notification`) e e-mail. Regras de produto em `Regras/04_AUDITORIA_TEMPO_E_METRICAS.md` e `Regras/06_NOTIFICACOES_E_COMUNICACAO.md`.

1. Pontos de entrada

Para	Use	Onde
Audit	<code>AuditService.log(user=, action=, activity=, task=, target_user=, field_name=, old_value=, new_value=, reason=)</code>	<code>audit/services.py</code> — único caminho de escrita; converte valores em texto
Notificar in-app	<code>NotificationService.notify(users=, event_type=, title=, message=, activity=, task=, url=, actor=)</code>	<code>notifications/services.py</code> — remove duplicados e grava em lote
Enviar e-mail	<code>EmailService.send_task_overdue</code> , <code>send_activity_approval_needed</code> , <code>send_client_information_request</code>	<code>notifications/services.py</code> — renderiza <code>templates/notifications/email/<nome>.txt</code> ; falha de envio é logada e não interrompe o fluxo
Resolver destinatários	<code>resolve_sector_members</code> , <code>resolve_sector_managers</code> , <code>resolve_sector_and_admins</code>	<code>notifications/recipients.py</code>

E-mails são **síncronos** (enviados durante a requisição) via `django.core.mail`. Não há fila.

2. Evento → auditoria → notificação → e-mail

Evento (serviço)	<code>AuditLog.Action</code>	<code>Notification.EventType</code> → destinatários	E-mail
Atividade criada/publicada	<code>CREATE</code>	<code>ACTIVITY_CREATED</code> → dono, criador	—
Campo de atividade editado	<code>UPDATE</code> (um por campo)	menções, se a descrição mudou	—
Troca de dono / assumir	<code>OWNER_CHANGED</code>	<code>OWNER_CHANGED</code> → antigo e novo dono	—
Finalizar como concluída	<code>COMPLETE</code>	<code>ACTIVITY_COMPLETED</code> → dono	—
Finalizar como cancelada	<code>CANCEL</code>	<code>ACTIVITY_CANCELLED</code> → dono	—
Reabrir	<code>REOPEN</code>	<code>ACTIVITY_REOPENED</code> → dono	—
Marcar pendente	<code>PENDENCY_OPENED</code> (+ <code>OWNER_CHANGED</code> se há aprovação)	<code>ACTIVITY_APPROVAL_NEEDED</code> → gestores do setor; <code>ACTIVITY_PENDING</code> → dono anterior (ou dono)	aprovação → gestores; pedido de informação → cliente, se marcado
Aprovar pendência	<code>PENDENCY_APPROVED</code>	<code>ACTIVITY_APPROVED</code> → dono	—
Anexo incluído/removido	<code>UPDATE</code> (campo "anexo")	—	—
Tarefa criada	<code>TASK_CREATED</code>	<code>TASK_ASSIGNED</code> → membros e gestores do setor	—
Tarefa editada	<code>UPDATE</code>	menções	—
Assumir / aceitar atribuição	<code>EXECUTOR_ADDED</code> (+ <code>ASSIGNMENT_ACCEPTED</code>)	<code>TASK_ASSIGNED</code> → quem aceitou	—
Atribuir a outra pessoa	<code>ASSIGNMENT_CREATED</code>	<code>TASK_ASSIGNMENT_PENDING</code> → atribuído	—

Evento (serviço)	Notification EventType →	destinatários	E-mail
Recusar atribuição	ASSIGNMENT_REJECTED	TASK_ASSIGNMENT_REJECTED → quem atribuiu	—
Remover participante	EXECUTOR_REMOVED	—	—
Trocar responsável	RESPONSABLE_CHANGED	TASK_RESPONSABLE_CHANGED → antigo e novo	—
Iniciar / retomar / tempo manual	SESSION_STARTED	—	—
Pausar	SESSION_PAUSED	—	—
Concluir tarefa	COMPLETE	TASK_COMPLETED → dono da atividade, responsável	—
Cancelar tarefa	CANCEL	—	—
Bloquear / desbloquear	BLOCK / UNBLOCK	TASK_BLOCKED / TASK_UNBLOCKED → dono da atividade	—
Enviar a outro setor	SECTOR_MOVED	TASK_ASSIGNED → novo setor	—
Devolver	RETURNED (+ SECTOR_MOVED)	TASK_RETURNED → os dois setores, dono	—
Reordenar fila	QUEUE_POSITION_CHANGED (por entrada afetada)	QUEUE_POSITION_CHANGED → dono da atividade	—
Propor prazo	DEADLINE_PROPOSED	DEADLINE_PROPOSED → dono	—
Aceitar prazo	DEADLINE_ACCEPTED	DEADLINE_ACCEPTED → quem propôs	—
Recusar prazo	DEADLINE_REJECTED + CONFLICT_OPENED	DEADLINE_CONFLICT → setor, dono	—
Resolver conflito	CONFLICT_RESOLVED	menções	—
Mensagem	—	MESSAGE_POSTED → envolvidos (menos autor e mencionados); MENTIONED → mencionados	—
Tarefa atrasada (check_overdue_tasks)	—	TASK_OVERDUE → setor, dono, responsável	sim
Segurança (AccessService, UserFormView)	PROFILE_CREATED, PROFILE_UPDATED, PROFILE_ACTIONS_CHANGED, PROFILE_ASSIGNED, PROFILE_REVOKED, ACTION_GRANTED, ACTION_REVOKED, SECTORS_CHANGED, USER_CREATED, PASSWORD_RESET (com target_user)	—	—

"Membros e gestores do setor" vem de `resolve_sector_and_admins`, que, apesar do nome, **não** inclui nenhum grupo de administradores.

Não auditado

Movimento de card no Kanban (atividade e tarefa), checklist, CRUD de cadastros, cores e status configuráveis, criação/publicação/versão de processo, autcadastro e confirmação de e-mail, troca de senha pelo próprio usuário.

`AuditLog.Action.SESSION_RESUMED` existe mas nunca é gravado (`resume` delega para `start`, que grava `SESSION_STARTED`).

3. Onde aparece

- **Histórico** (`/historico/`, `HistoryView`): `AuditLog` ligados a atividades da organização, com filtro `?event=` (tarefas, prazos, fila, devolucoes, tempo). Eventos de segurança sem atividade **não** aparecem ali. O texto de cada linha vem do filtro `{{ entry|audit_phrase }}` (`activities/templatetags/lps.py`).

- **Linha do tempo** da atividade/tarefa (`templates/activities/_timeline.html`): mesma fonte, filtrada pelo objeto.
- **Caixa de notificações** (`/notificacoes/`, `NotificationListView`):
 - abas `?filter=: acao` (padrão), `nao-lidas`, `mencoes`, `alertas`, `todas`; busca `q`, ordenação `ordem`, 30 por página;
 - cada evento é classificado em uma categoria (delegação, prazo, menção, alerta, conclusão, informativo);
 - a aba **Ação necessária** reconsulta o estado atual (`_still_needs_action`): some da lista quando a proposta de prazo foi decidida, o conflito resolvido, o bloqueio encerrado, a atribuição respondida etc.;
 - abrir uma notificação (`<pk>/read/`) marca como lida e só redireciona para destinos internos validados.
- **Contador** do sino: context processor `unread_notifications_count`.

09 — Rotas

Referência de todas as URLs. A coluna "Ação" indica a ação do catálogo verificada **na view** (`ActionRequiredMixin` ou `dispatch`); em `activities`, a maioria das verificações acontece dentro do serviço chamado — ver `06_ATIVIDADES_E_TAREFAS.md`. Toda view da LPS exige login e organização (`OrganizationRequiredMixin`), exceto login, cadastro, confirmação de e-mail e recuperação de senha.

1. Raiz (`config/urls.py`)

Caminho	Nome	Destino
<code>admin/</code>	—	Admin do Django
<code>accounts/login/</code>	<code>login</code>	<code>LoginView</code> + <code>EmailAuthenticationForm</code>
<code>accounts/logout/</code>	<code>logout</code>	<code>LogoutView</code>
<code>accounts/esqueci-senha/, .../enviado/</code>	<code>password_reset</code> , <code>password_reset_done</code>	views nativas
<code>accounts/redefinir-senha/<uidb64>/<token>/, .../concluido/</code>	<code>password_reset_confirm</code> , <code>password_reset_complete</code>	views nativas
<code>accounts/</code>		<code>accounts.urls</code>
<code>notificacoes/</code>		<code>notifications.urls</code>
<code>processos/</code>		<code>processes.urls</code>
<code>painel/</code>		<code>painel.urls</code>
<code>..</code>		<code>core.urls</code> , depois <code>activities.urls</code>
<code>media/<path></code>		<code>static.serve</code> de <code>MEDIA_ROOT</code> (sem login)
<code>atividade-arquivos/<path></code>		<code>static.serve</code> de <code>ACTIVITY_FILES_ROOT</code> (sem login)

2. `accounts` (`/accounts/`)

Caminho	Nome	Função
<code>me/</code>	<code>profile</code>	Meu perfil (telefone, setor principal)
<code>criar-conta/</code>	<code>signup</code>	Autocadastro
<code>confirmar-email/</code>	<code>verify-email</code>	Digitar o código
<code>confirmar-email/reenviar/</code>	<code>verify-email-resend</code>	Reenviar código (POST)
<code>confirmar-email/alterar/</code>	<code>verify-email-change</code>	Trocar o e-mail pendente

3. `core` (`/`)

APIs de busca (JSON, filtradas pela organização): `api/pessoas/` (`person-search`, aceita `?sector=`), `api/clientes/` (`client-search`), `api/setores/` (`sector-search`), `api/empresas/` (`company-search`), `api/obras/` (`site-search`), `api/centros-de-custo/` (`costcenter-search`), `api/tags/` (`tag-search`).

Cadastros

Caminho	View	Ação
cadastros/ (?tab=)	CadastroHomeView (cadastros)	—
cadastros/setores/novo/ , <pk>/	SectorFormView	setor.editar
cadastros/empresas/nova/ , <pk>/	CompanyFormView	empresa.gerir
cadastros/obras/nova/ , <pk>/	SiteFormView	obra.gerir
cadastros/centros-de-custo/novo/ , <pk>/	CostCenterFormView	centro_custo.gerir
cadastros/clientes/novo/ , <pk>/	ClientFormView (também em popup)	cliente.gerir
cadastros/motivos/novo/ , <pk>/	ReturnReasonFormView	motivo_devolucao.gerir
cadastros/estagios-de-tarefa/novo/ , <pk>/	TaskStageFormView	estagio_tarefa.gerir
cadastros/estagios-de-tarefa/reordenar/	TaskStageReorderView	estagio_tarefa.gerir
cadastros/tags/novo/ , <pk>/	TagFormView	tag.gerir
cadastros/<tab>/<pk>/situacao/	CadastroToggleActiveView (cadastro-toggle)	a da aba
cadastros/<tab>/<pk>/cor/	SwatchColorSaveView (cadastro-color-save)	a da aba

Configurações

Caminho	View	Ação
configuracoes/	SettingsView (settings) — preferências de notificação (guardadas na sessão)	—
configuracoes/estagios-de-atividade/novo/ , <pk>/	ActivityStageFormView	estagio_tarefa.gerir
configuracoes/status/<domain>/novo/ , <pk>/	WorkflowStatusFormView	cor_status.gerir
configuracoes/status/<domain>/<code>/editar/	EnumColorLabelFormView (enumcolor-label-edit)	—
configuracoes/cores/<domain>/salvar/	EnumColorSaveView (enumcolor-save)	—
configuracoes/cores/<domain>/restaurar/	EnumColorResetView (enumcolor-reset)	—
configuracoes/fluxo/<kind>/<pk>/excluir/	FlowConfigDeleteView (flow-config-delete)	depende do tipo

Usuários e segurança

Caminho	View	Ação
usuarios/	UserListView (user-list)	usuario.visualizar
usuarios/novo/ , <pk>/	UserFormView (user-create, user-edit)	usuario.editar
usuarios/<pk>/acessos/	UserAccessView (user-access)	seguranca.gerir_autorizacoes
usuarios/<pk>/acessos/<assignment_pk>/remove/	UserAccessRemoveView	seguranca.gerir_autorizacoes
usuarios/<pk>/concessoes/	UserGrantActionView (user-grant)	seguranca.gerir_autorizacoes
usuarios/<pk>/concessoes/<grant_pk>/remove/	UserGrantRemoveView	seguranca.gerir_autorizacoes
permissoes/	PermissionMatrixView (permissions)	seguranca.gerir_perfis
permissoes/<pk>/salvar/	PermissionUpdateView (permissions-update)	seguranca.gerir_autorizacoes
perfis/novo/ , <pk>/	ProfileFormView	seguranca.gerir_perfis

4. activities (/)

Início e visões gerais

Caminho	Nome	Função
..	home	"O que precisa da minha atenção agora?"
fila/, fila/<sector_pk>/	queue, queue-sector	Fila do setor
fila/entrada/<pk>/reordenar/		Reordenar (<code>fila.reordenar</code> no serviço)
gestao/	management	Gestão por exceção (<code>metricas.visualizar</code> em algum escopo)
historico/	history	Histórico de auditoria (<code>?event=</code>)

Atividades

Caminho	Função
atividades/ (activity-list)	Lista com abas e filtros
atividades/nova/ (activity-create)	Wizard passo 1
atividades/<pk>/nova/contexto/, .../detalhes/, .../descartar/	Wizard passos 2 e 3, descartar rascunho
atividades/nova-rapida/	Criação mínima (JSON)
atividades/busca/	Busca de atividades abertas (JSON)
atividades/kanban/, atividades/calendario/	Kanban por estágio, calendário por prazo
atividades/<pk>/ (activity-detail)	Ficha da atividade (rascunho redireciona ao wizard)
atividades/<pk>/painel/	Fragmento para painel lateral
atividades/<pk>/editar/, prazo/, dono/, assumir/	Editar, prazo, trocar dono, assumir
atividades/<pk>/finalizar/, concluir/, cancelar/, reabrir/	Encerramento e reabertura
atividades/<pk>/pendente/, pendencia/aprovar/	Pendência
atividades/<pk>/mensagem/, continuar/	Mensagem; comentário + anexo num envio
atividades/<pk>/anexos/, anexos/<attachment_pk>/remove/	Anexos
atividades/<pk>/mover-estagio/	Mudar coluna do Kanban (só <code>stage</code>)
atividades/<activity_pk>/tarefas/rapida/ (task-quick-create)	Nova tarefa na atividade (Ajax)

Tarefas

Caminho	Função
tarefas/ (task-list), tarefas/kanban/, tarefas/calendario/	Lista, Kanban, calendário
tarefas/nova-rapida/	Nova tarefa com seletor de atividade
tarefas/<pk>/ (task-detail), tarefas/<pk>/painel/, tarefas/<pk>/editar/	Ficha, painel lateral, edição
tarefas/<pk>/assumir/, iniciar/, pausar/, retomar/, concluir/, desbloquear/	Ações de execução (<code>task-assume</code> , <code>task-start</code> ...)
tarefas/<pk>/iniciar/ajax/, pausar/ajax/, concluir/ajax/	Mesmas ações em JSON, para o painel
tarefas/<pk>/bloquear/	Bloquear (<code>tarefa.bloquear</code> na view)
tarefas/<pk>/devolver/	Devolver (<code>tarefa.devolver</code> na view)
tarefas/<pk>/mover/	Outro setor (<code>tarefa.mover_setor</code> na view)

Caminho	Função
tarefas/<pk>/cancelar/	Cancelar (<code> tarefa.cancelar</code> na view)
tarefas/<pk>/executores/ , executores/<user_pk>/remove/	Participantes
tarefas/<pk>/alterar-responsavel/	Responsável
tarefas/<pk>/atribuicoes/<assignment_pk>/aceitar/ , .../recusar/	Responder atribuição
tarefas/<pk>/tempo/	Lançamento manual de tempo
tarefas/<pk>/mensagem/	Mensagem
tarefas/<pk>/mover-estagio/	Mudar coluna do Kanban (só <code>stage</code>)
tarefas/<pk>/checklist/ , checklist/<pk>/alternar/ , checklist/<pk>/remove/	Checklist
tarefas/<pk>/prazo/propor/ , prazos/<pk>/aceitar/ , prazos/<pk>/recusar/ , conflitos/<pk>/resolver/	Prazo e conflito

5. processes (/processos/)

` ( `process-list` ),  `novo/ (process-create) , / (process-edit) , /informacoes/ , /output/ , /inputs/ (+ /remove/) , /critérios/ (+ /remove/) , /fluxo/ (+ reordenar/ , /remove/) , /publicar/ , /nova-versao/ , /ativo/ . Autorização nos serviços (processo.*)`.

6. notifications (/notificacoes/)

Caminho	Nome	Função
``	<code>notification-list</code>	Caixa com abas (<code>?filter=</code>), busca e paginação
<pk>/read/	<code>notification-mark-read</code>	Marca lida e redireciona para um destino interno validado
<code>mark-all-read/</code>		Marca todas

7. painel (/painel/)

Telas em branco ("em construção") que mantêm o menu completo: `equipe/pessoas/` , `equipe/capacidade/` , `equipe/carga-de-trabalho/` , `filas-e-gargalos/filas/` , `.../gargalos/` , `.../bloqueios/` , `.../devolucoes/` , `processos/modelos/` , `insights/` , `desenvolvimento/` , `resultados/` , `integracoes/` , `configuracoes/motivos-de-bloqueio/` , `configuracoes/motivos-de-devolucao/` .

Telas reais, com views de `core` :

Caminho	Nome	Ação
<code>configuracoes/etapas-e-status/</code>	<code>config-etapas-status</code>	<code>estagio_tarefa.gerir</code> ou <code>cor_status.gerir</code> , conforme a aba
<code>configuracoes/prioridades/</code>	<code>config-prioridades</code>	<code>cor_prioridade.gerir</code>

Para listar as rotas de verdade a qualquer momento:

```
python manage.py shell -c "from django.urls import get_resolver; [print(p) for p in get_resolver().reverse_dict.keys() if isinstance(p, str)]"
```

10 — Frontend

A interface é renderizada no servidor com templates do Django. CSS e JavaScript são arquivos estáticos simples — **não há package.json, bundler nem etapa de build**. As telas funcionam sem JavaScript sempre que possível; o JS melhora a experiência (arrastar, modais, busca), mas envia os mesmos formulários que já existem na página. Identidade visual em `Telas/Elementos/ e Telas/MENU_E_SUBMENUS_LPS.md`.

1. Templates (`templates/`)

Pasta / arquivo	Conteúdo
<code>base.html</code>	Casca da aplicação: menu lateral (recolhível), barra superior com cronômetro e sino, mensagens, blocos <code>{% block %}</code>
<code>_icon.html</code> , <code>_icons.html</code> , <code>_logo.html</code>	Ícones SVG e logo, incluídos com <code>{% include %}</code>
<code>accounts/</code>	Login, criar conta, confirmar e-mail, trocar e-mail, perfil, recuperação de senha
<code>activities/</code>	Início, listas, Kanban, calendários, ficha da atividade, ficha da tarefa, wizard, fila, gestão, histórico
<code>core/</code>	Cadastros, configurações, usuários, permissões, etapas e status, prioridades
<code>notifications/</code>	Caixa de notificações, textos de e-mail em <code>email/*.txt</code>
<code>processes/</code>	Lista, criação e edição de processo
<code>painel/em_construcao.html</code>	Tela em branco dos itens de menu ainda não construídos
<code>registration/</code>	Assunto e corpo do e-mail de recuperação de senha

Convenção: arquivos que começam com `_` são **parciais** incluídos em outras telas ou devolvidos por Ajax — ex.:

`activities/_activity_drawer.html`, `_task_drawer.html`, `_task_checklist.html`, `_timeline.html`, `_tasks_block.html`, `_activity_kanban_card.html`. `activities/_form_fields.html` renderiza qualquer formulário campo a campo e é reutilizado por várias telas.

As pastas `templates/demands/` e `templates/workflows/` estão vazias (restos).

2. Context processors

Disponíveis em todo template:

Processor	Variáveis	Uso
<code>notifications.context_processors.unread_notifications_count</code>	contador de não lidas	sino
<code>acessos.context_processors.navigation</code>	<code>lps_nav</code> (flags <code>management</code> , <code>cadastros</code> , <code>users</code> , <code>security</code>), <code>lps_org</code> , <code>lps_open_tasks</code> , <code>nav_active</code> , <code>nav_active2</code> , <code>nav_cadastros_tab</code>	o que o menu mostra e qual item está ativo — só UX
<code>activities.context_processors.my_active_sessions</code>	<code>my_active_sessions</code> , <code>my_active_sessions_count</code>	cronômetro da barra superior

3. Template tags ({% load lps %})

Todos são **filtros**, em `activities/templatetags/lps.py`:

Filtro	O que faz
<code>entry\ audit_phrase</code>	Converte um <code>AuditLog</code> em frase em português ("iniciou a tarefa")
<code>name\ field_label</code>	Nome amigável de um campo auditado
<code>timedelta\ duration_hm</code>	<code>4h20</code> , <code>35min</code>
<code>user\ avatar_color</code>	Cor do avatar (uma de 6, por <code>pk</code>)

4. JavaScript (`static/js/`)

Arquivo	Papel
<code>shell.js</code>	Casca: menu no celular, janelas de ação
<code>modal.js</code>	Busca um formulário existente via <code>fetch</code> e abre por cima da tela
<code>task-drawer.js</code>	Painel lateral da tarefa sobre a ficha da atividade
<code>checkboxlist.js</code>	Checklist da tarefa (página e painel), sem recarregar
<code>timer.js</code>	Cronômetro da sessão; conta localmente, o servidor é a referência
<code>queue.js</code>	Arrastar para reordenar a fila (preenche o formulário de posição existente)
<code>task-kanban.js</code> , <code>activity-kanban.js</code>	Kanban com arrastar; ao soltar, só atualiza <code>stage</code>
<code>task-stage-reorder.js</code>	Reordenar estágios de tarefa
<code>person-picker.js</code> , <code>person-multi-picker.js</code>	Seletor de pessoa (responsável) e de várias pessoas (participantes), usando <code>api/pessoas/</code>
<code>tag-picker.js</code>	Seletor de tags com chips
<code>mention.js</code>	Autocomplete de <code>@menção</code> em campos marcados
<code>rich-text.js</code>	Editor de descrição (<code>contenteditable</code>); o HTML é sanitizado no servidor por <code>core/sanitize.py</code>
<code>form-summary.js</code>	Resumo ao lado do formulário, atualizado ao digitar
<code>color-utils.js</code> , <code>color-palette-picker.js</code>	Paleta de 36 cores (espelha <code>core/colors.py</code>) e seletor
<code>notifications.js</code>	Evita que o cache do navegador mostre contador/lista antigos
<code>auth.js</code>	Mostrar/ocultar senha nas telas de conta

Ao mudar a paleta, altere **os dois**: `core/colors.py` e `static/js/color-utils.js`.

5. CSS e imagens

- `static/css/app.css`: todo o estilo da aplicação.
- `static/css/auth.css`: telas de conta (login, cadastro, confirmação).
- `static/img/`: favicon e logos da LPS (completo, completo escuro, símbolo).

Em produção os arquivos são coletados por `collectstatic` em `staticfiles/` e servidos pelo `whitenoise` com nome versionado por hash — ao editar um CSS/JS, basta fazer deploy; não é preciso trocar a URL no template (use sempre `{% static %}`).

11 — Testes

Onde estão os testes, como rodá-los e como escrever novos seguindo o padrão do projeto.

1. Onde estão

Testes do Django ficam em cada app (`tests.py` e, em `activities`, também `test_views.py`):

Arquivo	Testes (aprox.)	Foco
<code>activities/tests.py</code>	71	Serviços: dono único, executores e tempo, sessões, devolução, fila, prazos, pendências, checklist, isolamento entre organizações
<code>activities/test_views.py</code>	71	Views: permissões por escopo, telas, Ajax, redirecionamentos
<code>acessos/tests.py</code>	36	Motor de autorização: escopos, perfis, concessões, teto de tenant
<code>core/tests.py</code>	28	Cadastros, busca de pessoas, formulário de usuário
<code>notifications/tests.py</code>	23	Destinatários, categorias, "ação necessária"
<code>accounts</code> , <code>audit</code> , <code>processes</code>	0	<code>tests.py</code> vazio

Além disso, `tests/checklist.test.cjs` testa o componente JavaScript do checklist (seção 4).

2. Rodando

```
python manage.py test # tudo
python manage.py test activities # um app
python manage.py test activities.test_views # um módulo
python manage.py test activities.test_views.TaskChecklistViewTests # uma classe
python manage.py test core.tests.UserFormAjaxTests.test_non_ajax_request_still_redirects # um teste
```

Opções úteis: `-v 2` (nome de cada teste), `--parallel` (mais rápido), `--keepdb` (reaproveita o banco de teste entre execuções).

Os testes usam `config.settings.dev` (via `manage.py`) com um banco SQLite em memória criado e destruído a cada execução — o seu `db.sqlite3` não é tocado. A suíte completa leva alguns minutos, porque todas as migrations rodam no início.

Falhas conhecidas

`core.tests.UserFormAjaxTests.test_ajax_request_returns_json_without_redirect` e `test_non_ajax_request_still_redirects` falham: o payload de teste não envia `password1` / `password2`, que são obrigatórios ao criar usuário. O teste está desatualizado, não o código. Ver [13_PENDENCIAS_CONHECIDAS.md](#).

3. Escrevendo testes

O padrão é criar organização, usuários e permissões explicitamente no `setUp`:

```
from django.contrib.auth import get_user_model
from django.test import TestCase

from acessos import catalog
from acessos.testing import grant_action
from core.models import Organization, Sector

User = get_user_model()

class MinhaFeatureTests(TestCase):
    def setUp(self):
        self.org = Organization.objects.create(name="Biasi")
        self.sector = Sector.objects.create(organization=self.org, name="Compras")
        self.user = User.objects.create_user("ana", email="ana@example.com", password="x")
        self.user.profile.organization = self.org
        self.user.profile.save(update_fields=["organization"])
        grant_action(self.user, catalog.TAREFA_CRIAR, organization=self.org)
```

Helpers de `acessos/testing.py`:

Helper	Uso
<code>ensure_catalog()</code>	Grava o catálogo de ações no banco de teste
<code>grant_action(user, key, organization=None, scope=None, sector=None, relation=None)</code>	Concessão direta; escopo da organização por padrão, ou de setor/relação
<code>grant_actions(user, keys, **kwargs)</code>	Várias de uma vez
<code>make_profile(organization, name, keys)</code>	Cria um perfil de acesso com as ações
<code>assign_profile(user, profile, sector=None, relation=None)</code>	Atribui o perfil com escopo

Dicas:

- Para testar **negação**, crie o usuário sem conceder a ação e confira que o serviço levanta `ActivityError` (ou a view responde 403).
- Para testar **isolamento**, crie uma segunda organização e confira que o recurso dela não aparece / dá 404.
- Para views com login: `self.client.force_login(self.user)`.
- Views Ajax: envie `HTTP_X_REQUESTED_WITH="XMLHttpRequest"`.
- Um usuário sem organização (`User.objects.create_user(...)` sem ajustar o perfil) serve para testar o redirecionamento do `OrganizationRequiredMixin`.

4. Teste JavaScript do checklist

`tests/checklist.test.cjs` renderiza o template real `activities/_task_checklist.html` (chamando o Python do `.venv`, ou o da variável `PYTHON`) e executa `static/js/checklist.js` no jsdom. Precisa de Node 18+; o jsdom é instalado numa pasta temporária, sem criar `package.json` no projeto:

```
$checklistDeps = Join-Path $env:TEMP 'lps-checklist-test-deps'
npm install --prefix $checklistDeps --no-save --package-lock=false --ignore-scripts jsdom@26.1.0
$env:NODE_PATH = Join-Path $checklistDeps 'node_modules'
node --test tests/checklist.test.cjs
```

Rode a partir da raiz do repositório. O jsdom não valida layout nem o envio por Enter do navegador — confira também manualmente (ver `tests/README.md`).

12 — Deploy no Render

A aplicação é publicada no Render pelo Blueprint `render.yaml`, no plano free: um Web Service (`lps-web`) e um PostgreSQL (`lps-db`). Este documento explica o arquivo, o primeiro deploy e as limitações do plano gratuito.

1. `render.yaml`

Chave	Valor	Por quê
<code>databases[0]</code>	<code>lps-db</code> , free, banco <code>lps</code> , usuário <code>lps</code>	Postgres gerenciado
<code>services[0].type / runtime / plan</code>	<code>web / python / free</code>	
<code>buildCommand</code>	<code>pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py migrate && python manage.py ensure_superuser</code>	Instala, coleta estáticos para o whitenoise, aplica migrations e cria o superusuário (se configurado)
<code>startCommand</code>	<code>gunicorn config.wsgi:application</code>	<code>wsgi.py</code> já usa <code>config.settings.prod</code>
<code>DJANGO_SETTINGS_MODULE</code>	<code>config.settings.prod</code>	Necessário porque o build usa <code>manage.py</code> , que sozinho usaria dev
<code>SECRET_KEY</code>	<code>generateValue: true</code>	O Render gera um valor aleatório no primeiro deploy
<code>ALLOWED_HOSTS</code>	<code>.onrender.com</code>	Qualquer subdomínio do Render
<code>CSRF_TRUSTED_ORIGINS</code>	<code>https://*.onrender.com</code>	Formulários sob HTTPS
<code>DATABASE_URL</code>	<code>fromDatabase: lps-db</code>	String de conexão injetada pelo Render
<code>PYTHON_VERSION</code>	<code>3.12.7</code>	Versão do Python no build
<code>DJANGO_SUPERUSER_USERNAME/EMAIL/PASSWORD</code>	<code>sync: false</code>	Preenchidos à mão no dashboard; nunca vão para o Git

As configurações de produção que tornam isso possível estão em `config/settings/prod.py`: whitenoise com `CompressedManifestStaticFilesStorage` e `SECURE_PROXY_SSL_HEADER` (o Render termina o TLS no proxy; sem essa linha, `SECURE_SSL_REDIRECT` entra em loop).

2. Primeiro deploy

- Render → **New** + → **Blueprint** → conecte o repositório `Lucasrafael703/workflow`. O Render lê o `render.yaml` e cria banco e serviço.
- No serviço `lps-web` → **Environment**, preencha `DJANGO_SUPERUSER_USERNAME`, `DJANGO_SUPERUSER_EMAIL` (obrigatório na prática: o login é por e-mail) e `DJANGO_SUPERUSER_PASSWORD`. Salvar dispara novo deploy, e o build cria o usuário.
- Entre em `https://<serviço>.onrender.com/admin/` com esse e-mail e senha.
- O banco de produção começa **vazio de dados de negócio**. Pelo Admin:
- crie a **Organização**;
- em **Accounts** → **Perfis**, vincule seu usuário a ela.
- O build **não** roda `seed_acoes` nem `seed_lps_demo`. Sem o catálogo de ações no banco, não dá para montar perfis de acesso para usuários comuns (o superusuário funciona porque ignora o motor). Até isso entrar no build (ver [13_PENDENCIAS_CONHECIDAS.md](#)), a forma de popular é adicionar temporariamente `&& python manage.py seed_acoes` (ou `seed_lps_demo --org-name "..."`) ao `buildCommand`.

Deploys seguintes acontecem a cada push na branch `main`.

3. E-mail em produção

Nenhuma variável `EMAIL_*` está no `render.yaml`, então produção usa o backend SMTP com host vazio. Enquanto não configurar (`EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `DEFAULT_FROM_EMAIL` no dashboard):

- **Criar conta** dá erro 500: o código de confirmação é enviado por `send_mail` sem tratamento de erro (`accounts/views.py`), depois que o usuário inativo já foi gravado;
- **Esqueci minha senha** mostra a tela de "enviado", mas o e-mail não sai (o Django só registra a falha no log);
- os e-mails de notificação (`EmailService`) também só registram a falha no log, sem quebrar a tela.

4. Limitações do plano free

Limitação	Efeito na LPS
Serviço hiberna após ~15 min sem acesso	Primeira requisição depois disso demora para responder
Postgres free expira (é apagado) após o período de gratuidade	Será preciso recriar o banco e rodar as migrations de novo
Disco não persistente	Anexos (<code>ACTIVITY_FILES_ROOT</code>) e mídia enviados somem a cada deploy/reinício
Sem Shell (recurso pago)	Não dá para rodar <code>manage.py</code> manualmente — por isso o superusuário é criado no build
Sem Cron Job gratuito	<code>check_overdue_tasks</code> não roda sozinho; avisos de atraso não são disparados

5. Python local × Render

O desenvolvimento local usa Python 3.14; o Render, 3.12.7 (fixado em `PYTHON_VERSION`). As versões em `requirements.txt` foram escolhidas para ter wheels nas duas (por exemplo, `psycopyg[binary]==3.2.13`; a 3.2.3 não tinha wheel para 3.14). Ao atualizar dependências, confira se a versão nova tem wheel para as duas versões de Python — ou alinhe `PYTHON_VERSION` com a versão local.

13 — Pendências conhecidas

Defeitos, lacunas e dívidas técnicas encontrados ao documentar o código (29/09/2026). Cada item foi conferido no código-fonte. Nada aqui foi corrigido — a lista serve para priorizar. Ao resolver um item, remova-o daqui.

Legenda de severidade: **Alta** (segurança ou quebra em produção) · **Média** (comportamento errado para o usuário) · **Baixa** (limpeza).

1. Segurança

#	Sev.	Onde	Problema
S1	Alta	config/urls.py	/atividade-arquivos/<path> e /media/<path> são servidos por <code>django.views.static.serve</code> sem exigir login nem autorização . Quem souber (ou adivinhar) <code><empresa>/<código ATV-...>/<arquivo></code> baixa o anexo. Solução típica: uma view que carrega o <code>ActivityAttachment</code> , verifica acesso à atividade e devolve o arquivo.
S2	Média	acessos/catalog.py × views	Ações de visualização nunca são verificadas: <code>atividade.visualizar</code> , <code> tarefa.visualizar</code> , <code> auditoria.visualizar</code> , <code> fila.visualizar_posicao_propria</code> . A ficha da atividade, a da tarefa e o histórico (<code>/historico/</code>) são limitados só pela organização — qualquer pessoa da organização abre qualquer atividade pelo link.
S3	Média	acessos/catalog.py × views	Outras ações sem verificação: <code>usuario.criar</code> (só decide se o botão "criar pessoa" aparece; <code>/usuarios/novo/</code> exige <code>usuario.editar</code>), <code>usuario.inativar</code> , <code>setor.criar</code> , <code>processo.aplicar</code> .
S4	Média	activities/views.py (<code>ActivityMoveStageView</code> , <code>TaskMoveStageView</code>)	Mover card no Kanban não verifica ação nenhuma e não é auditado — qualquer pessoa da organização muda o estágio de qualquer atividade/tarefa.

2. Defeitos

#	Sev.	Onde	Problema
D1	Alta	acessos/services.py, <code>ResourceContext.of</code> (ramo <code>site</code>)	Lê <code>resource.company_id</code> , mas <code>core.Site</code> não tem mais <code>company</code> (migration <code>core/0010_remove_site_company_site_client</code>). Verificar autorização passando uma obra como recurso levanta <code>AttributeError</code> .
D2	Alta (prod)	accounts/views.py, <code>_send_verification_email</code>	<code>send_mail</code> sem tratamento de erro. Sem SMTP configurado, "Criar conta" dá 500 depois de já ter gravado o usuário inativo.
D3	Média	activities/services.py, <code>return_task</code>	Grava <code>DEVOLVIDA</code> e logo chama <code>move_to_sector</code> → <code>enqueue</code> , que sobrescreve para <code>EM_FILA</code> . Nenhuma tarefa fica <code>DEVOLVIDA</code> ; a notificação <code>TASK_RETURNED</code> sai da aba "Ação necessária" imediatamente (ela checa <code>status == DEVOLVIDA</code>).
D4	Média	activities/services.py, <code>TaskService.block</code>	Só recusa se a tarefa já está bloqueada; permite bloquear tarefa concluída ou cancelada .
D5	Média	activities/services.py	Atividade nunca passa de <code>ABERTA</code> para <code>EM_ANDAMENTO</code> pelo trabalho (iniciar tarefa só marca <code>first_action_at</code>). <code>EM_ANDAMENTO</code> só vem de reabertura ou aprovação de pendência.
D6	Média	activities/services.py	Pendência sem aprovação (<code>MATERIAL</code> , <code>INFORMACOES_CLIENTE</code>) não tem como ser resolvida: a atividade fica <code>PENDENTE</code> até ser finalizada.
D7	Baixa	activities/models.py, <code>TaskResponsavelChangeLog</code>	Dois <code>__str__</code> ; o segundo (vence) usa <code>self.user</code> , que não existe — <code>str()</code> levanta erro (aparece no Admin). Parece ter sido escrito para <code>TaskExecutor</code> , que ficou sem <code>__str__</code> .
D8	Baixa	activities/models.py	<code>Activity.Status.BLOQUEADA</code> e <code>Task.Status.NAO_INICIADA</code> existem, mas nenhum fluxo os usa. <code>AuditLog.Action.SESSION_RESUMED</code> nunca é gravado.
D9	Baixa	core/tests.py, <code>UserFormAjaxTests</code>	2 testes falham: o payload não envia <code>password1</code> / <code>password2</code> , obrigatórios ao criar usuário. O teste está desatualizado.

3. Funcionalidades incompletas

#	Onde	Situação
F1	<code>processes/</code>	Processos são criados, versionados e publicados, mas não são aplicados a atividades: nada preenche <code>Activity.process_version</code> , gera tarefas das etapas ou grava <code>ActivityInputValue</code> / <code>ActivityCriterionCheck</code> .
F2	<code>notifications/recipients.py</code> , <code>ADMIN_GROUP_NAME</code>	A setting é lida mas não usada; <code>resolve_sector_and_admins</code> devolve só membros e gestores do setor. O help de <code>check_overdue_tasks</code> ainda fala em "administradores".
F3	Autocadastro	Conta criada em "Criar conta" nunca recebe organização pelo app; é preciso o Admin do Django. A tela <code>/usuarios/</code> só lista quem já está na organização.
F4	<code>painel/</code>	Vários itens do menu são telas em branco (equipe, filas e gargalos, insights, resultados, integrações...).
F5	Escalonamento de prazo	Conflitos só são registrados e notificados; o motor de escalonamento é D1 (Regras 10 §57).

4. Deploy

#	Onde	Situação
P1	<code>render.yaml</code>	O build não roda <code>seed_acoes</code> : em produção o catálogo de ações fica vazio e não dá para montar perfis de acesso. Adicionar <code>&& python manage.py seed_acoes</code> ao <code>buildCommand</code> (é idempotente).
P2	<code>render.yaml</code>	Sem variáveis de e-mail: ver D2 e 12_DEPLOY_RENDER.md .
P3	Plano free	Anexos se perdem a cada deploy; <code>check_overdue_tasks</code> não roda (sem cron nem Shell).

5. Limpeza do repositório

#	Item
L1	<code>demands/</code> , <code>workflows/</code> , <code>templates/demands/</code> , <code>templates/workflows/</code> : restos de apps removidos (fora de <code>INSTALLED_APPS</code> , sem código).
L2	<code>venv/</code> : ambiente virtual quebrado de outra máquina (aponta para <code>C:\Users\saulo\...\Python313</code>). Ignorado pelo Git, mas <code>.claude/launch.json</code> ainda usa <code>venv/Scripts/python.exe</code> — deveria ser <code>.venv</code> .
L3	<code>tests/README.md</code> e o cabeçalho de <code>tests/checklist.test.cjs</code> mandam rodar "a partir da pasta <code>workflow</code> " (nome antigo); o certo é a raiz do repositório.
L4	<code>Regras/12_...</code> e <code>Regras/13_...</code> têm "11 —" no título, inconsistente com o nome do arquivo.
L5	Docstring de <code>ActivityAttachmentService</code> cita <code>MEDIA_ROOT</code> ; o storage real é <code>ACTIVITY_FILES_ROOT</code> .
L6	<code>Activity</code> , <code>Task</code> e <code>DeadlineConflict</code> ainda declaram <code>Meta.permissions</code> do Django (<code>can_change_owner</code> etc.), sem uso — a autorização é o motor do <code>acessos</code> .